

TokTier: Exact Stateful CPU+GPU Tokenization for Agentic LLM Serving

Zhenyu Zhang
Arizona State University
Tempe, Arizona, USA
zzhan641@asu.edu

Zhichao Cao
Arizona State University
Tempe, Arizona, USA
Zhichao.Cao@asu.edu

Abstract

LLM serving systems cache prompt KV state, yet most front ends still re-tokenize the full request text on every call. The mismatch is most costly for coding agents, whose sessions repeatedly submit a long transcript after appending a small tool result. However, reusing prior tokenization results is hard because even a short append can change token boundaries near the end of the previous sequence. Across 153,951 calls from two agent ecosystems, the median call appends only about 1.4 K characters, and only 1.0–3.6% of calls start or rebuild a session. Those calls, however, carry full contexts that reach millions of characters. At the fleet level, the aggregate prompt-cache hit rate is 94.1%, and as it approaches 0.99, tokenization grows from 10% to 64% of time to first token (TTFT) in our component measurements.

We present TOKTIER, a stateful tokenization service for this two-mode workload with CPU and GPU integration. TOKTIER enforces one contract: emitted token IDs from TOKTIER are always identical to full reference tokenization of the request text. For a session continuation, TOKTIER keeps the session’s previous token sequence, re-tokenizes a small window around the append, and accepts the splice only when a per-request check finds a stable pre-tokenization boundary. A failed check triggers a wider window or full reference tokenization. For a call without a reusable prefix, TOKTIER decomposes GPT-family regex pre-tokenization into run-local rules and executes exact pre-tokenization and BPE on a GPU for low latency and high throughput. A sampled shadow verifier re-checks live traffic against the reference.

Across 17 production tokenizer families, our differential campaigns include 1.50×10^{10} split checks, full sweeps of a 12.4 TB real-text corpus, and 93,000+ replayed agent steps, all with zero divergence. Incremental repair takes 0.5–1.1 ms from 100 K to 3 M characters, up to **437**× faster than HF tokenization and 2.1× faster at 1 M characters than the strongest cache-based baseline (i.e., Gigatoken) in its most favorable, fully prewarmed mode. GPU full tokenization encodes a 1 M-character request in 0.87 ms, up to **491**× below HF tokenization and 23.4× below the fastest previously published CPU method on the same texts and protocol. With vLLM in the loop, TOKTIER lowers median TTFT by 16–34% in loaded regimes and P99 TTFT by 23% under recorded burst arrivals. Under a 50 ms P99 objective, a four-core CPU-based repair

pool plus one GPU sustains 1,821 requests/s, where a 16-core stateless CPU front end saturates at only 40 requests/s.

1 Introduction

A coding agent works through a sequence of LLM model calls. It reads a file, edits it, runs a tool, observes the result, and decides what to do next [33, 35]. Each call carries the session transcript, and each tool result extends that transcript before the next call goes out. A single user instruction can fan out into dozens of calls. The calls arrive at machine cadence, and the transcript grows from turn to turn.

This access pattern exposes a mismatch in current LLM serving stacks. Prefix caching (i.e., KV cache) lets the model reuse KV state for the unchanged part of a prompt, but a stateless front end still converts the complete request text into token IDs on every call. In the coding-agent traffic we measure, the fleet-level prompt-cache hit rate (cached input tokens as a share of all input tokens, pooled over the fleet) is 94.1%. Moreover, the median call appends about 1.4 K characters to a context of 86 K to 123 K tokens. The model may process only the small uncached suffix via the KV cache, while the tokenizer scans the entire transcript. As the prompt-cache hit rate approaches 0.99, tokenization grows from 10% to 64% of time to first token (TTFT) in our component sweeps (§2).

This type of workload has two distinct modes. On the one hand, most requests are session continuations. They extend a session whose token sequence was produced moments earlier. On the other hand, a small share, 1.0–3.6% in our traces, carries no reusable prefix. For example, a session starts, a compacted history is rebuilt, or a request reaches a worker that holds no session state. Such calls are rare, but they carry the full context and arrive in bursts. Those two modes ask for different execution strategies to achieve high throughput and low latency during tokenization: 1) incremental repair should do work proportional to the newly appended text; and 2) full tokenization should process a large, previously unseen context with a low tail latency.

However, reusing token IDs across turns is harder than reusing a byte prefix. Tokenization is not compositional. For strings A and B , $\text{tok}(A) \parallel \text{tok}(B)$ can differ from $\text{tok}(A \parallel B)$. The pre-tokenizer may move a piece boundary when B arrives, and BPE may then choose different merges inside the piece. For example, a boundary inside the word “pipeline”

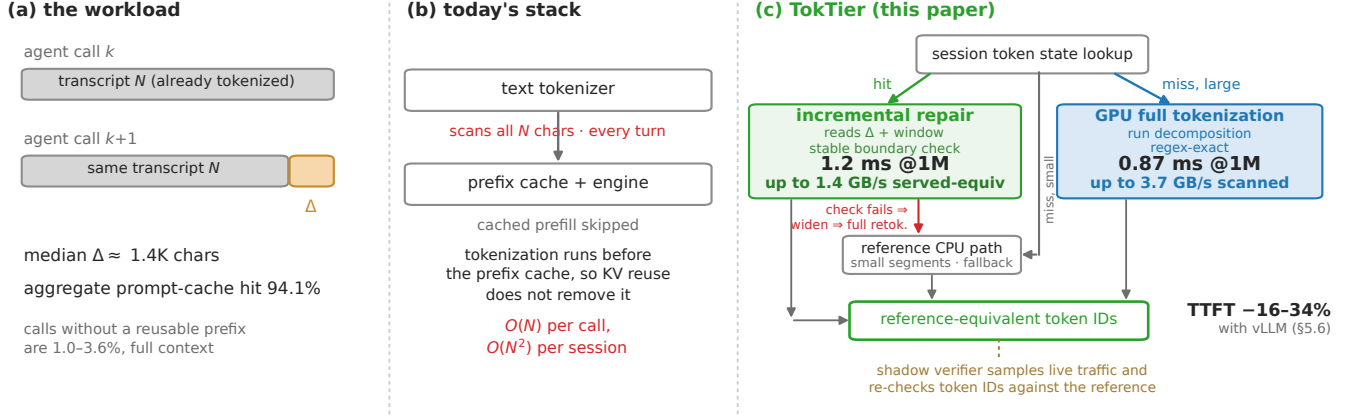


Figure 1. One agent turn through today’s stack and through TokTier. The request appends a median $\sim 1.4\text{K}$ characters to a transcript that was tokenized on previous turns, and today’s front end re-scans the complete context before the prefix cache can act. TokTier repairs session continuations around the append, routes rare initializations and rebuilds to an exact GPU path or the reference CPU path, and samples all outputs through a shadow verifier. Latencies are measured P50 at 1 M-character contexts, and the two bold figures are each path’s best measured single-request throughput under the served-equivalent and scanned accountings, which are never mixed (§5.3).

turns one reference token into two different tokens (§3.1). Fixed overlap heuristics do not solve the problem, because some tokenizer rules let a change propagate past any chosen radius. To ensure LLM serving performance and accuracy, a production system must know when a cached prefix is safe to splice, and it must fall back when that condition cannot be established.

Moreover, calls without a reusable prefix pose a separate problem. GPT-family tokenizers begin with leftmost-first regex matching, followed by BPE [26]. The regex is sequential in its usual form, because every match begins where the previous match ends. Therefore, leveraging GPU or other hardware to achieve highly parallelized tokenization for high throughput and low latency while guaranteeing correctness is very challenging. Existing GPU tokenizers obtain parallelism by relaxing this stage, by targeting a different scheme, or by checking only against their own CPU implementation [11, 13, 36]. Token IDs are both model input and prefix-cache keys, so a single changed ID can alter model behavior and silently invalidate KV cache reuse.

To address those challenges, we designed and implemented TokTier around these two modes with one correctness contract: for every request, the emitted IDs must equal the IDs produced by a frozen reference tokenizer (e.g., HF tokenizer) on the complete text. TokTier stores each live session’s token IDs and byte spans. On a session append, it re-tokenizes the new text plus a small suffix of the old context. It compares the fresh and cached token records, and splices only when the matched region passes a *stable boundary check*, a per-request test that the pre-tokenizer provably cannot see across. When the check fails, TokTier widens the window

and eventually runs the reference tokenizer on the full request. An unsuccessful repair therefore costs latency without changing the output, and it is rare. In replayed agent traffic, 56,049 of 56,052 appends splice on the first window, three widen it once (0.005%), and none falls back to full retokenization (§5.3).

For a call without a reusable prefix, TokTier uses a GPU tokenizer derived from an equivalent representation of GPT-family pre-tokenization. First, the input is classified into maximal character-class runs. Piece starts then follow from the position within a run, a bounded amount of neighboring text, and a few run-level summaries. This *run decomposition* removes the sequential regex scan while preserving its output, and a GPU BPE pipeline encodes the resulting pieces. In addition, small segments stay on the CPU, and every implementation binds to a content-addressed tokenizer registry. Finally, a background verifier samples live requests and compares their IDs with the reference implementation. This guard covers bugs that depend on execution history and therefore escape fresh-process tests.

The evaluation asks whether the design is exact, whether its two paths match the measured workload, and whether the front-end gain survives contact with a serving engine. We examine 17 tokenizer families with version-pinned artifacts. Differential campaigns include synthetic and adversarial inputs, full sweeps of a 12.4 TB real-text corpus, and 93,000+ replayed agent steps. The tested configurations produce no divergence from the reference. Incremental repair stays at 0.5–1.1 ms from 100 K to 3 M characters. It is up to 437 $\times$ faster than HF tokenization and 2.1 $\times$ faster than the strongest cache-based alternative tokenizer (i.e., Gigatoken [27]) at

1 M characters and $3.0\times$ at 2 M. The GPU path sustains 3.8–4.7 GB/s and encodes a 1 M-character full context in 0.87 ms, $23.4\times$ below the fastest previously published CPU method. With vLLM in the loop, TOKTIER reduces median TTFT by 16–34% in loaded regimes and P99 by 23% under recorded burst arrivals. A four-core repair pool and one GPU sustain 1,821 requests/s under a 50 ms P99 objective, compared with 40 requests/s for a 16-core stateless CPU front end (§5).

This paper makes three major contributions: 1) to the best of our knowledge, it is the first to characterize session-level tokenization in coding-agent traffic, revealing that the stream consists of frequent small updates alongside rare, large rebuilds (§2); 2) it presents an exact stateful CPU+GPU tokenization service, up to $437\times$ and $491\times$ faster than HF tokenization on its two paths, incorporating checked boundary repair for session continuations and a run-local reformulation of GPT-family pre-tokenization that enables full GPU tokenization (§3, §4); and 3) it develops a validation methodology that combines per-request checks, version-pinned differential testing, real-text sweeps, and runtime sampling, then evaluates the complete service in front of vLLM (§5).

Our implementation is available at <https://github.com/asu-idi/toktier>.

2 Background and Workloads

LLM serving studies usually summarize a request by prompt length, output length, and arrival time. Tokenization needs one more distinction. It must separate the text already seen in the current session from the text that has just arrived. We write N for the complete context, Δ for the new text, and h for the prompt-cache hit ratio reported by the serving API. Full re-tokenization reads N characters. Ideal incremental repair would read close to Δ while returning exactly the same token IDs.

2.1 Where tokenization sits

An LLM serving front end receives text, applies added-token handling, normalization, pre-tokenization, and subword encoding, then sends token IDs to the model engine [9, 14]. Prefix caching (i.e., KV cache) begins after those IDs are provided to the inference engine [14, 38]. A high KV-cache hit ratio therefore removes the redundant model-side prefill work; it does not, however, remove front-end work.

The output of a tokenizer must also match the reference tokenizer used by the model during training and previous serving, because token IDs are part of the model input and most prefix caches index their entries by token sequence [38]. An approximate tokenizer changes the token stream the model sees, which can significantly influence the serving accuracy and reduce KV cache reuse. Published GPU tokenizers accept that trade (§6). We use the reference tokenizer’s full-text output as the fundamental contract throughout this paper and guarantee exact matches in TOKTIER.

2.2 Trace sources

Our primary dataset contains 153,951 calls from ten months of day-to-day Claude Code and Codex CLI use by six users on nine machines. A local collector parses the agents’ session logs, and exports counts only (Appendix B lists the fields). Text is length-counted in memory and discarded, and identifiers are HMAC-hashed with keys that never leave the source machine. The parser also detects duplicated and replayed log records. This check removed 26,578 phantom calls, 14.7% of one ecosystem’s parsed data. Collection, consent, and parsing details appear in Appendix B.

We check these traces against three other independent sources. Provider-side usage metadata for the same fleet covers ~ 5.12 billion tokens of the same kind of traffic. Here and throughout, the hit rate of a call is the share of its input tokens the API reports as read from the prompt cache, and a token-weighted aggregate sums both counts over all calls before dividing. On that accounting, the fleet hit rate is 94.1%. A public autonomous-agent trace (codex_swebenchpro) contains 20,230 calls from 610 successful SWE-Bench Pro trials [7, 10], whose publishers report an aggregate hit rate of 94.2% for that corpus. Its release preserves message structure and message lengths, and substitutes length-preserving filler for redacted spans (Appendix B.3). The TraceLab corpus [40] contains 357 K steps from 4,265 Claude Code and Codex sessions collected at another institution. TraceLab omits text, so it cannot replay tokenization, but it checks context size, append size, hit rate, and pacing externally (Appendix B). The sources share no collection pipeline and no users. Where they overlap, they agree.

2.3 Most calls add little text to a large context

Figure 2 summarizes the common case. The median append is about 1.4 K characters in the interactive traces, with P90 near 9–14 K. The public autonomous trace is heavier, with a 3.8 K-character median, but it has the same shape. Median context size is 86 K (Codex) to 123 K (Claude Code) tokens and extends toward a million tokens. At the same time, 74–87% of calls have $h > 0.9$, and the median call has h between 0.98 and 0.99.

These values expose the work amplification of full re-tokenization. TraceLab’s median step carries 126,180 cached-prefix tokens and 857 appended tokens. Its per-step ratio of complete context to append has a median of 132 and a token-weighted aggregate of 23.5. The same token-weighted ratio is about 10 on our per-call traces and about 17 at the fleet’s 94.1% hit rate. Current text interfaces therefore process one to two orders of magnitude more material than the request adds.

Moreover, the agent calls also form deep loops. One user turn produces 3–5 model calls at the median, 27–34 at P90, and 87–103 at P99. The autonomous trace runs 30 calls per trial at the median. The tokenization cost is paid at every step.

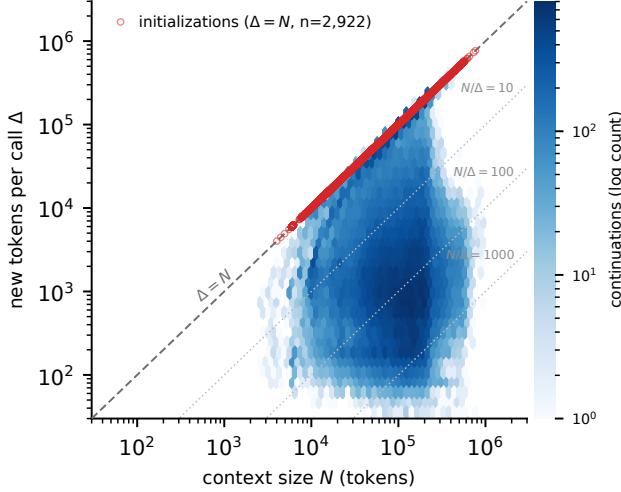


Figure 2. Joint distribution of context size N and new tokens per call Δ over the 153,951 interactive calls, in the API’s token accounting. Session continuations concentrate one to three orders of magnitude below the diagonal, so most calls add little text to a large context. Session initializations and rebuilds sit on the $\Delta=N$ diagonal and carry complete contexts up to 10^6 tokens. Marginal distributions and per-source detail are in Fig. 15 of Appendix B.

Thus, an $O(N)$ tokenizer accumulates $O(N^2)$ work over a session whose context grows monotonically.

2.4 Session initializations and rebuilds are rare but large

In trace terms, a full-context call arrives with no cached prefix ($h=0$) and a session continuation arrives with part of its prompt already cached. In tier terms, a session state miss occurs when a call reaches a worker with no reusable token state. This happens in several cases, for example, at session start, after history compaction, or after a migration. Full-context calls account for only 1.0–3.6% of calls pooled by trace source. However, their sizes are usually very large, differing from the typical append by two orders of magnitude. They carry complete contexts of 10^4 to 10^6 tokens and often arrive together, when many sessions start or rebuild at once.

2.5 Session state can outlive KV state

Human pauses are long relative to default prompt-cache lifetimes. Median gaps between user turns are 3.6–6.4 minutes in our traces. About 55% of Claude Code gaps and 42% of Codex gaps exceed the 5-minute default cache TTL. TraceLab shows the same decay from the provider side, where the mean cached share of a step falls from 0.96 at sub-minute pauses to 0.17 beyond an hour (Appendix B). Token IDs and byte spans cost tens of bytes per token, far less than KV state.

A tokenization service can therefore retain session state after the engine evicts the corresponding KV blocks, with a very small memory overhead. When the session returns, the service repairs the token sequence in milliseconds even if the model must rebuild part of its KV cache. Section 5.7 measures the memory cost of this state and the session state hit rate that longer retention buys (Figure 14).

2.6 Tokenizer design requirements

The aforementioned measurements and analysis lead to **four design requirements** for the new tokenizer: 1) the common path must keep per-session token state and make its work follow the change rather than the complete context; 2) full tokenization must absorb large requests without creating a latency tail; 3) both paths must support multiple frozen tokenizer versions, because 12+ model versions appear simultaneously in our traces. And 4) every path must return the IDs exactly the same as the reference token sequence, since the emitted IDs can influence both LLM output accuracy and KV cache effectiveness. This last requirement rules out boundary heuristics and approximate GPU tokenizers even when their average throughput is high.

The absolute CPU cost of tokenization is manageable in some present deployments, but its trend is unfavorable. Under the measured mixture, full re-tokenization on a modern Rust CPU-based tokenizer costs 13.4 ms of core time per request, which reads as 6.7 front-end cores per 1,000 GPUs at 0.5 requests/s/GPU (Appendix D). Three trends multiply that number: 1) faster model inference raises requests completed per GPU, 2) longer contexts increase full-tokenization time, and 3) rising KV cache hit rates remove model-side latency while leaving front-end work unchanged (Appendix B.6). We treat the trend as motivation rather than as a fleet-size prediction. Section 5 measures the current service-time, tail, and capacity effects directly.

3 TOKTIER System Design

TOKTIER is a tokenization service placed between the request router and the model engine. It accepts request text and a model identifier, then returns exact reference-equivalent token IDs. The service keeps token state for live sessions and selects an execution path based on the state and the request size. Figure 3 shows the data and workflow.

Each tokenizer version is registered by content hash. Its entry carries the added-token rules, normalization configuration, pre-tokenization rules, vocabulary, merge table, and the family-specific checks used by the repair and GPU paths. A request is never interpreted through ambient process state, and a session created under one tokenizer version is never repaired under another.

The router classifies each request by session state. A *session state hit* finds a compatible session record, one whose text prefix matches the new request. The record holds token

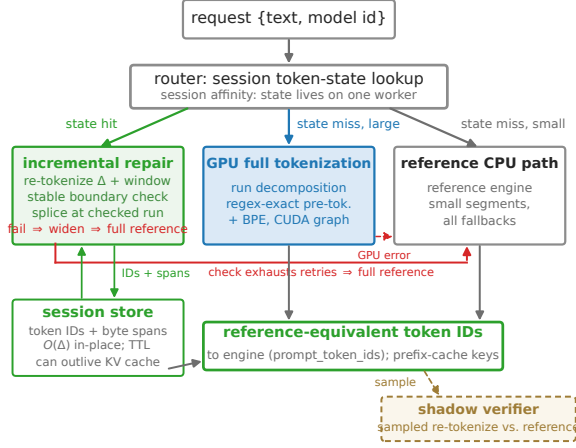


Figure 3. Request lifecycle through TokTier. The router checks for live session token state, then sends session continuations to boundary repair, large state-miss segments to the GPU path, and small segments or any fast-path failure to the reference CPU path. The session store keeps token IDs and byte spans per live session. A shadow verifier re-tokenizes a sampled fraction of emitted IDs against the reference engine and quarantines divergences.

IDs and source byte spans from the previous call, and TokTier repairs this record near the change. If a *session state miss* finds no compatible record, the request is a session initialization or a history rebuild. Therefore, its context is tokenized in full. The router decides the request state inside the tokenization tier, independently of the engine-side prompt-cache reuse h of §2. The two layers hold separate state with separate lifetimes. In TokTier, large state-miss segments go to the GPU path of §4. Small segments use the reference CPU tokenizer, where kernel-launch overhead would dominate if we still used the GPU. Every path is held to one output contract: **the emitted token IDs are always identical to full reference tokenization of the request text**. Every mechanism below may fail toward more work, never toward different IDs.

3.1 Why an append can change old tokens

Modern BPE tokenizers apply two stages. A pre-tokenizer divides text into short *pieces*, usually with a regex, and BPE then encodes each piece independently. Thus, an arbitrary text boundary is not a token boundary.

Figure 4 gives a real example. The previous request ends after “pipe”, and the next request appends “line”. The reference tokenizer sees “pipeline” as one piece and emits one vocabulary token. Tokenizing the two sides independently emits “pipe” and “line” as two tokens. The appended text changes the trailing token of the old prefix and shifts every later position. The drift will influence the LLM output accuracy and silently invalidate KV cache reuse for the rest of

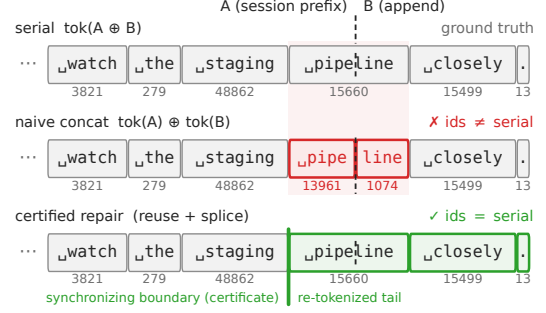


Figure 4. A boundary inside the word “pipeline” changes the token sequence (Llama-3.1-8B tokenizer, real token IDs below each box). Independent tokenization produces two tokens (pipe+line) where full tokenization produces one (pipeline). Repair re-tokenizes the affected region and reuses the cached prefix only after finding a stable boundary, reproducing the serial stream bit for bit.

the session (since the token-ID sequence is the prefix-cache key).

Two mechanisms create this problem. First, when text arrives on the right, the pre-tokenizer can move a piece boundary. Second, the BPE merge order inside the resulting piece can then change. A fixed overlap radius around the append is not a correctness rule. Digit grouping, white-space lookahead, and newline absorption can push the effect past any chosen radius, and our adversarial oracle produced cascading counterexamples for every bounded-radius rule we formulated. We therefore use a window only to *search* for a reusable boundary, and base acceptance on a separate per-request check derived from the tokenizer family. The published track record shows the difficulty is routinely underestimated. LoPT, the one peer-reviewed segmented tokenizer, specifies an experimental configuration that does not satisfy its safety theorem’s stated precondition [28]. Gigatoken, a high-throughput open-source engine, documents boundary stability as an engineering assumption. Our full-corpus differential evaluation over 11 tokenizer families finds mismatches against the reference implementation at a rate of 1.3×10^{-8} (558 divergent document-tokenizer pairs in 4.18×10^{10} comparisons); every archived specimen traces to a difference in the Unicode data versions the two engines carry rather than to segmentation, which is precisely the kind of version-relative discrepancy an assumption-based design cannot certify away [27].

3.2 Incremental repair

Incremental repair updates the previous token sequence instead of rebuilding it. Suppose a session stored the tokenization of text A and the next request is $A \parallel B$. The service takes

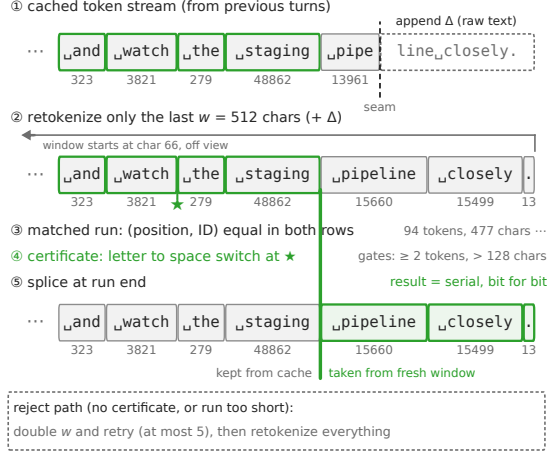


Figure 5. Incremental repair on the running example of Figure 4 (same frozen tokenizer, every token ID is real engine output). The service re-tokenizes the append and a suffix of the stored context, matches fresh and cached token records, checks that the equal run contains a stable pre-tokenization boundary (the starred class transition, called a certificate in the formalization of Appendix A), and splices at the end of that run. A failed check widens the window and eventually falls back to full reference tokenization.

the last w characters of A , with $w = 512$ by default, appends B , and tokenizes this window with the same reference-compatible engine used for the session. It then compares the fresh records with the cached records that overlap the old part of the window.

Figure 5 shows the full operation. TOKTIER finds the longest run of records on which the cached and fresh sequences agree in both position and token ID. The *stable boundary check* then admits the run through three conditions: 1) the run must contain at least two tokens; 2) it must cover more normalized characters than the longest token the vocabulary can emit (probed per tokenizer; 128 for the Llama family), which rules out an accidental single-token match; and 3) most importantly, the run must contain a family-specific stable boundary, a character-class transition at which the pre-tokenizer’s output to the right provably does not depend on text to the left. A matched ID run alone is not sufficient, because context-dependent digit grouping defeats it, a failure our certificate-level oracle exposed adversarially. Appendix A formalizes the stable boundary as a *synchronizing boundary* and proves the splice theorem.

When the conditions hold, cached records are kept before the matched run and fresh records after it. The implementation splices at the end of the equal run, which Corollary A.6 shows is interchangeable with splicing at the stable boundary inside it. The resulting ID sequence equals full tokenization of $A \parallel B$. When a condition fails, TOKTIER doubles w and

retries, at most five times, then sends the complete request to the reference engine. The search can miss an available boundary, but it cannot accept an invalid token sequence output. A miss therefore costs extra work and never changes the returned IDs. The same procedure applied on both sides of an edit handles mid-context mutations, and large rewrites that fail both searches are tokenized in full. Misses are rare in practice: in replayed agent traffic, 56,049 of 56,052 appends splice on the first window, three widen the window once, and none reaches the full-retokenization fallback (§5.3).

3.3 Why the splice check is sufficient

The formal argument is short at the system level. The pre-tokenizer maps the input into an ordered sequence of pieces, and BPE encodes each piece without state from other pieces. At a stable boundary, the piece sequence to the right is independent of the left context. If the cached and fresh records agree across a run that contains such a boundary, both executions have reached the same piece sequence and the same BPE output. Cached records can therefore serve the left side, and freshly tokenized records serve the right.

The proof is parameterized by a frozen tokenizer configuration. We establish which character-class transitions synchronize each supported family and check that the configuration satisfies the proof assumptions. 15 of the 17 families we examined meet these conditions. The other two are provably outside the predicate class (one normalizer erases whitespace structure entirely, leaving no internal boundary to repair against) and always take the full-retokenization path.

Although the theorem covers the family-level boundary abstraction, it does not prove the evolving Rust, Python, and CUDA implementation. We validate the implementation with version-pinned differential campaigns against the reference tokenizer, and we sample deployed outputs at runtime. Section 5.2 separates these forms of evidence and reports their coverage, and Appendix D places the two nearest neighbors on the resulting guarantee ladder. In general, across all differential campaigns and deployed sampling to date, this validation has observed zero divergence on the shipped configurations (§5.2).

3.4 State management

The session store keeps token IDs, byte spans, the tokenizer hash, and a compact index from text positions to token records in memory. Repair edits these arrays in place, and derived indices are updated lazily, so the bookkeeping cost follows the repair window rather than the full context. An earlier implementation rebuilt offset arrays on every turn and silently recovered $O(N)$ behavior even though tokenization touched only the window. With it fixed, the protocol does $O(\Delta + w)$ work end to end, independent of N .

TOKTIER reclaims the state at session end. Therefore, memory usage depends only on live sessions rather than content

history, and it can achieve a longer lifetime than KV cache (§2.5). A tokenizer worker owns the sessions assigned to it. Requests that lose affinity become session state misses and remain correct. Cross-worker state transfer is an optimization that the current system does not require (planned in our future work).

4 Exact GPU Tokenization

Session initializations and history rebuilds contain no reusable session state. Their contexts are large enough to trigger long CPU service times, leading to high latency, especially when several sessions start or rebuild together. To address this challenge, TOKTIER moves full tokenization to a GPU for high-performance parallel processing, which also guarantees exact equivalence to the reference token sequence.

4.1 Removing the sequential regex scan

GPT-family tokenizers specify pre-tokenization as a leftmost-first alternation regex with backtracking [26]. In the direct implementation, each match begins at the previous match’s end. This dependency prevents independent matching of arbitrary chunks for parallel processing, and splitting the text first would recreate the seam problem of §3.1. Prior GPU tokenizers avoid the tension by weakening this stage or by targeting a different specification [11, 13, 36]. Therefore, none of them reports reference-exact token-ID agreement, which our output contract requires.

Our central observation is that the production patterns we study never needed a backtracking engine. We classify each character into one of four classes (letter, number, white-space, other) and form maximal same-class runs. Whether a character begins a piece then depends only on its offset within the run, at most four characters of lookback across the run boundary, and three per-run aggregates (run start, first non-CRLF position, last CRLF position). Character classification can be applied in parallel, run summaries come from prefix scans, and the piece-start test is an independent per-character predicate (Figure 6).

We derive the rules one regex alternative at a time. The cl100k lineage maps directly onto the four classes and local run rules. The o200k lineage adds a case-boundary rule and a sparse contraction path, and Unicode marks take a narrow sparse fallback while the common path stays vectorized. DeepSeek-family tokenizers [5, 6] apply three splitters in sequence. Their composition folds into the same per-character predicate, provided each splitter re-splits every piece of the previous stage and unmatched spans survive as implicit pieces.

The derivation makes the regex-to-run-rule mapping concrete in two steps. First, for each alternative of the reference regex, it names the run rule that reproduces that alternative’s matches. Second, it pins the tables under which this correspondence holds. One is the tokenizer configuration, frozen

by content hash. The other is the set of Unicode character-class tables, probed directly out of the reference engine rather than taken from library data (§5.2). A change to either table therefore invalidates the derivation instead of silently changing its meaning.

Differential testing then checks the implementation at two levels, the piece boundaries produced by pre-tokenization and the final token IDs (§5.2). Both levels are needed, because the ID level can hide a boundary failure. Suppose a faulty rule splits one piece into two halves and no merge in the vocabulary spans the split point. BPE then encodes the two halves into the same ID sequence that the intact piece would produce, and an ID-level comparison passes on a wrong piece division. The same fault becomes visible only when an input happens to place a mergeable pair across that seam, so an ID-only harness finds it by accident rather than by construction. Boundary-level comparison removes that dependence on the input.

4.2 GPU BPE

After pre-tokenization, each piece is encoded independently. Merge ranks, vocabulary entries, and byte strings are packed into GPU hash tables. Two equivalences license a parallel schedule. A round’s minimum merge rank identifies one pair value, and merging all leftmost non-overlapping occurrences of that pair equals iterated single merges. Only pairs adjacent to a completed merge must be probed again.

Work is dispatched by piece length, so that each piece is encoded by the smallest execution unit able to hold its state. Pieces up to 32 bytes, the vast majority in natural text, run thread-per-piece with the sequence in registers. Pieces of 33–128 bytes run warp-per-piece: the 32 lanes probe candidate pairs in parallel, a shuffle reduction finds the minimum rank, a ballot materializes the hit positions as a bitmask, and the leftmost non-overlapping selection is then evaluated in closed form on that mask. Longer pieces fall to a block-per-piece kernel with a shared-memory candidate bitmap. Families configured with `ignore_merges` (Llama 3, gpt-oss) admit one further shortcut: their reference encoder emits a single ID whenever the whole piece is already a vocabulary entry, so the kernel probes the vocabulary first and skips the merge loop on a hit.

The complete pre-tokenization path consists of on-device UTF-8 decoding, character classification, run construction, the rule predicate, and two prefix scans. Multi-document batches concatenate inputs and cut runs at document boundaries, at 0.36 μ s per small document. Pre-tokenization alone reaches 30–77 GB/s on one RTX PRO 6000, depending on script mix. The complete path, including BPE and output compaction, sustains a multi-document batch throughput of 3.8–4.7 GB/s (876–1206 Mtok/s) across English, CJK, and templated corpora, byte-identical to the reference.

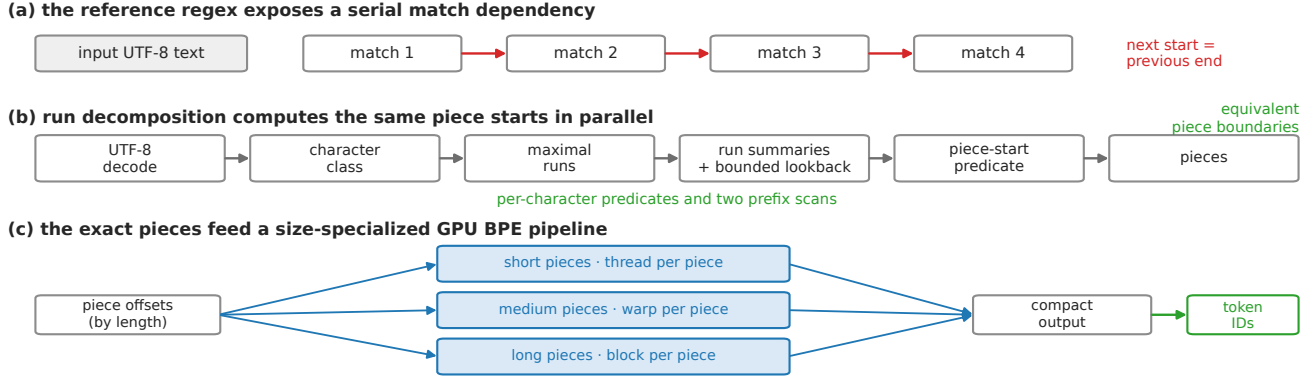


Figure 6. Run decomposition. The reference regex exposes a serial dependency between matches. The equivalent formulation computes character classes, maximal runs, and a local piece-start predicate with parallel passes. The resulting pieces feed size-specialized BPE kernels.

4.3 Single-request path

Batch throughput does not determine the latency of a single state-miss request. A conventional GPU pipeline reads intermediate counts back to the host before choosing the next launch. Those synchronizations are invisible in batch benchmarks and only visible in a single request, especially when the host is busy. TOKTIER keeps piece counts, dispatch lists, and output length in device memory. Kernel geometry depends on buffer capacity rather than host-visible counts, and the whole bytes-to-IDs chain is captured as a CUDA graph over a small set of input-size buckets. Host synchronizations drop from eight per request to two. Under contention from 28 competing CPU processes, the graph path keeps P99 near its idle value, moving from 0.38 to 0.49 ms on a 50 K-character request, while the eager pipeline’s P99 doubles from 0.52 to 1.05 ms (§5.4).

4.4 Scope and fallbacks

The GPU path covers the tokenizer body for the cl100k, o200k, and DeepSeek pattern families evaluated in this paper. Added-token literal extraction runs before family dispatch. NFC normalization stays on the CPU for the one family that requires it. The GPU path currently returns IDs without source byte spans, so a state-miss request that must initialize session state for later repair uses the reference tokenizer to obtain spans. Unsupported tokenizers and requests that fail a family check remain on the CPU reference path. In addition, segments above a 2 KB threshold go to the GPU while smaller ones stay on the CPU. The router also spills to the CPU under GPU backpressure. These cases may change latency and capacity, but the output contract is unchanged.

4.5 Service implementation

The session store and repair logic are implemented in Rust. Tokenizer entries are immutable after registration, and workers use session affinity so a session continuation reaches the state its previous turn created. Added and special tokens are literal strings that the model reserves as single IDs, `<|endoftext|>` for example. A leftmost-longest literal pass extracts them before the tokenizer family handles the remaining segments.

Reference CPU path. This path is the frozen HuggingFace reference implementation. It serves the small segments, unsupported families, repair fallback, backpressure spill, and state rebuilds that need source spans. It carries under 0.3% of request characters across our serving mixes, so reference speed suffices. The tier also ships the fastest available Rust tokenizer, but strictly as a timing baseline. Our verifier caught exactly this class of engine returning history-dependent IDs (§5.8), so speed and correctness anchoring are assigned to different engines by construction. The repair-window engine itself is pluggable. We run the same protocol unchanged over the Python reference stack and a Rust crate, with the certification replay campaigns of §5.2 as the admission bar.

Engine interface. TOKTIER submits token IDs through vLLM’s `prompt_token_ids` interface. We verified that prefixes inserted through text and through IDs share the same vLLM prefix-cache key space. Text-written prefixes are hit by ID-submitted requests at a 99.84% rate (99.94% in reverse) with identical outputs, so the engine needs no modification. Instrumenting vLLM 0.25 also located the original tokenization work in the API-server process before engine scheduling, which is why removing it changes TTFT.

Shadow verification. A background thread samples emitted sequences and re-tokenizes the same text with the reference engine. The verifier compares IDs exactly and quarantines any mismatch with a content hash for offline reproduction. If it falls behind, it drops samples and increments a visible counter. The sampling rate is 5% in the serving experiments and 100% in offline sweeps. Runtime sampling covers history-dependent implementation failures that no fresh-process test can exercise (§5.8).

5 Evaluation

We first present the overall comparison against every measured baseline, then test both execution paths against frozen reference tokenizers. We measure incremental repair and GPU full tokenization separately, followed by evaluating burst tails, capacity, and TTFT with vLLM. The final experiments cover resource use and runtime verification. What limits the GPU path is discussed with the other limitations in §7.

Experiments run on a dual-socket AMD EPYC 9115 host (32 physical cores, SMT disabled) with four RTX PRO 6000 Blackwell GPUs of 96 GB each. CPU experiments are NUMA-pinned. One GPU serves vLLM 0.25 with prefix caching enabled and one serves TokTier unless stated otherwise. Tokenizer artifacts, datasets, and dependency versions are frozen by content-addressed manifests. CPU one-shot latency uses one measured encode per process, because Rust tokenizers retain word caches across calls.

The reference is the HuggingFace fast tokenizer for each frozen model. The main CPU performance baseline is *fastoken* [8], a separately distributed Rust tokenizer that vLLM supports as an opt-in backend. We also compare TokTier with Gigatoken [27], a cache-based high-performance CPU tokenizer, and with our clean-room LoPT reproduction [28]. Approximate GPU tokenizers appear only in correctness comparisons, because they do not satisfy the output contract.

5.1 Performance overview

Figure 7 places TokTier against every measured baseline before the detailed protocols. On session continuations, TokTier repair prices one append at 0.5–3.5 ms P50 from 100 K to 4.4 M characters and stays near flat, while every full re-tokenizer grows with the context. The strongest cache-based baseline, Gigatoken, wins below its 100 K–500 K crossover and loses by a growing margin beyond it. On session initializations, the GPU path answers a fresh request in 0.29–3.59 ms over the same span, at least 7× below every CPU full-tokenization baseline at every shape. In bulk full tokenization, one GPU sustains 3.8–4.7 GB/s, against 1.35–2.06 GB/s for the strongest 32-core CPU configuration and 0.69–0.85 GB/s for Gigatoken batch on an empty cache, while

Table 1. Main correctness campaigns. Split rows compare every pre-tokenization boundary, and end-to-end rows compare final token IDs. Every comparison is against the reference implementation.

Path	Inputs	Checks	Div.
GPU split-level	synthetic, adversarial, and a full 12.4 TB real-text sweep; four pattern families	1.50×10^{10}	0
GPU end-to-end	six production tokenizers; synthetic, adversarial, and real documents	6.21×10^7	0
Incremental repair	two public agent corpora and 15,000 adversarial edits	1.09×10^5	0
Shadow sample	offline, capacity, and vLLM runs	$> 5 \times 10^4$	0*

*the verifier also exposed one genuine bug in an external tokenizer (§5.8).

one TokTier repair core serves up to 1.4 GB/s of context under the separate served account. The rest of this section establishes exactness first, then details the protocols and baselines behind each line (§5.3, §5.4).

5.2 Exactness

Table 1 summarizes the differential campaigns. Every admitted configuration reports zero divergence. The split-level campaigns are multiplicative rather than sharded. Each of the four GPU-path pattern families ran its own sweep of the 12.4 TB corpus (Nemotron-CC v2.1, distributed as 4.59 TB of compressed archives, and corpus sizes throughout this paper are decompressed text), every document for two families and 3.7×10^9 documents each for the other two.

Split-level testing found an implementation bug that final IDs did not expose reliably. An o200k fallback path missed a run-head condition and could fuse two pieces across a chunk seam. A six-character input reproduces the failure. Smaller synthetic suites had passed clean, and the bug appeared within one minute of the million-input mixed campaign. We fixed it and added regression pins.

Real text found a different class of problem. Character-class tables that three components had derived from stdlib data (Unicode 15.0) disagreed with the reference engine’s own tables (16.0) on ~ 9.7 K codepoints. Synthetic generators built from the older tables could never emit those characters, and two hours of CommonCrawl-derived text exposed the skew that 6×10^7 synthetic checks could not. Every class table is now probed directly out of the reference engine. The corpus-scale end-to-end comparator matched the unmodified reference encoder bit for bit on 3.7×10^6 documents before it judged anything, and it caught all planted faults used to validate the harness. The DeepSeek group (V3, V4-flash, HY3) then passed the same ladder with zero divergence. That

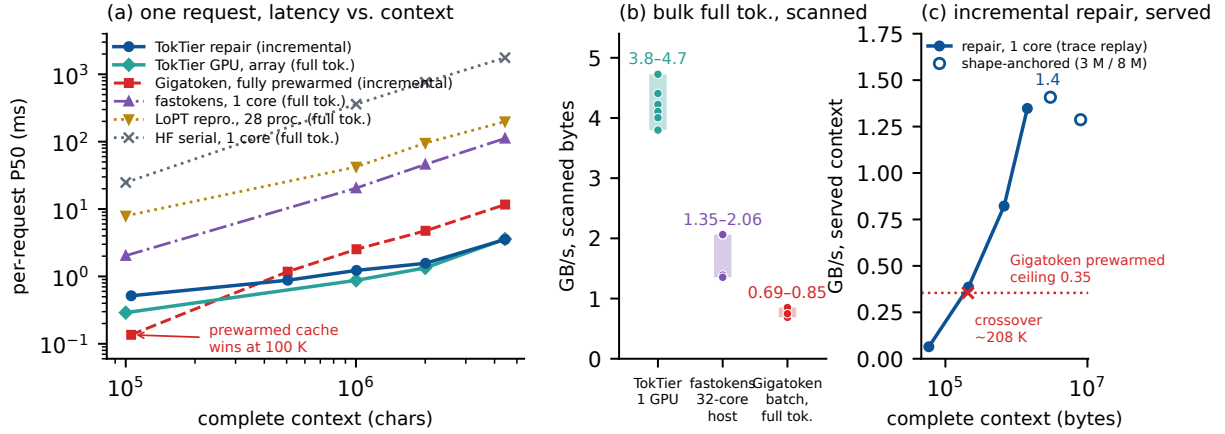


Figure 7. Performance against every measured baseline. (a) Single-request P50 latency versus complete context on identical real texts. Incremental-repair lines price one append on a live session under the protocol of \$5.3, and full-tokenization lines price a full encode of a fresh request under the protocol of \$5.4. Gigatoken runs in its most favorable, fully prewarmed mode and wins below the 100 K–500 K crossover. (b) Bulk full tokenization in the scanned-bytes account, with per-corpus measured points. The Gigatoken column is its batch mode on the same host with an empty cache. (c) The served-context account for one incremental-repair core versus complete context, on the public-trace replay of Appendix D, with shape-anchored points at the 3 M and 8 M shapes. The dotted line is Gigatoken’s prewarmed ceiling in the same served account, crossed near 208 K bytes. The scanned and served accounts sit on separate axes and are never mixed (§5.3).

ladder comprises 10⁷ synthetic split checks, end-to-end suites on all three variants and dispatch paths (1.28×10^5 checks), and one full-corpus split sweep of every document (3.80×10^9 checks). The three variants share the splitter configuration hash, so one sweep judges the group.

Incremental repair replays 74,064 repairs from public SWE-smith trajectories and 19,620 from the public autonomous-agent trace, with zero ID divergence. Re-replaying both corpora under the shipped boundary-check configuration accepts every splice the length-only check accepts, 92,484 certified splices with zero violations and indistinguishable latency. It also applies 15,000 adversarial mid-context edits targeting digit runs, ideograph–punctuation seams, and fraction characters, with identical behavior. Unsupported families and failed boundary checks take the fallback path and are excluded from the accepted count.

5.3 Incremental repair: latency and throughput

Figure 8 measures 12,674 real append steps under the o200k family. The shipped Rust store, which replays the certification battery of §5.2 with zero divergence, keeps median TokTier repair latency between 0.5 and 1.1 ms from 100 K to 3 M characters. Full re-tokenization on fasttokens with a prewarmed word cache grows with the complete context and loses in every bucket. TokTier repair is 1.7× faster below 100 K characters, 4.1× from 100 K to 300 K, 7.2× from 300 K to 1 M, and 13× from 1 M to 3 M. The comparison leans the baseline’s way, since fasttokens re-timed with an empty word cache runs 2.6–3.0× slower than the archived curve we plot.

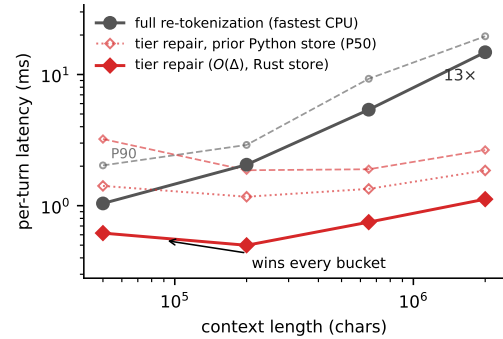


Figure 8. Incremental repair latency versus complete context length on the public-trace replay (log–log, solid P50, dashed P90). The Rust session store keeps TokTier repair latency at 0.5–1.1 ms from 100 K to 3 M characters, while full CPU re-tokenization grows with context size. The dotted line is the prior Python-store implementation on the same protocol.

The previous Python session store measured 1.2–1.9 ms with mild growth, because it copied arrays during each splice. In-place Rust bookkeeping recovered the intended $O(\Delta + w)$ scaling.

Fast-path coverage is a separate question from speed, so we instrument the same replays for per-splice window outcomes (Figure 9). The first 512-character window accepts 99.995% of 56,052 real splices, across append sizes from tens of characters to 213 K. Three splices widen the window once,

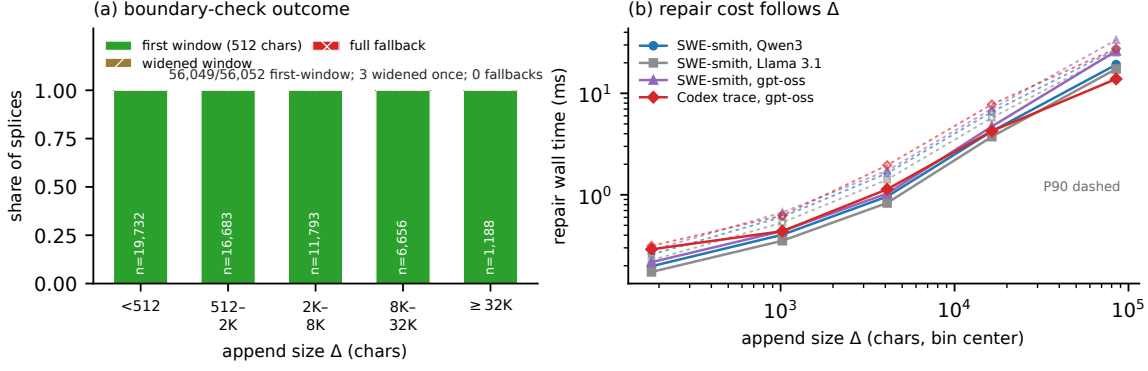


Figure 9. Boundary-check behavior over 56,052 replayed real splices (SWE-smith streams under three families, plus the public Codex trace). (a) The default 512-character window accepts 56,049 splices on the first attempt at every append size, three splices widen the window once, and none falls back to full retokenization. (b) TokTier repair wall time grows with the append and not with the context, as $O(\Delta + w)$ predicts. Latency here is replay-measured in one process and supports shape comparisons only.

Table 2. Session-continuation append latency (ms) on identical Qwen3 session texts, same host. One timed call per sample, $n=24$ per shape, median append 1.5–1.6 K characters. Context sizes (ctx) are in characters. Gigatoken runs in its most favorable mode, with its per-object cache fully prewarmed on the session prefix. The 3 M and 8 M shapes of the same protocol appear in the text and in Appendix C.

ctx	repair (ours)		Gigatoken prewarmed		HF serial
	P50	P90	P50	P90	
100 K	0.52	0.87	0.14	0.22	23.6
500 K	0.88	1.68	1.18	1.25	165.2
1 M	1.23	2.83	2.53	2.64	318.0
2 M	1.57	4.60	4.76	5.25	658.8
4.4 M	3.54	10.01	11.66	12.11	1548.0

and none reaches the full-retokenization fallback. Adversarial inputs can force the fallback by construction. The observed agent traffic does not.

Table 2 prices one continuation append on identical real session texts. Gigatoken wins at 100 K characters, and the crossover lies between 100 K and 500 K. At 1 M characters repair is $2.1\times$ faster at the median, at 2 M it is $3.0\times$ faster, and the lead widens with context ($3.4\times$ at 3 M, $3.3\times$ at 4.4 M). Gigatoken’s latency keeps growing because even a full cache hit rescans the complete context, while TokTier repair scans the append and the window. Its cache is also process-lifetime memory keyed on content, where TokTier repair state is per-session and freed. Fasttokens takes 9.7 ms at 1 M characters on the same inputs and stays 7.9–16 $\times$ slower than TokTier repair from 1 M upward. On an 8 M-character headroom shape beyond any deployed window (median 2.04 M tokens), TokTier repair still leads at P50 (6.25 vs. 23.42 ms), and the first repair after each session bootstrap forms the entire P90 tail at 26–32 ms. Appendix C sweeps the append size from

1 K to 100 K characters at every shape. TokTier repair cost grows with the appended bytes at 2.9–3.9 MB/s, as $O(\Delta)$ predicts, and the fully prewarmed cache catches up only past the measured append distribution’s 99th percentile of 38 K characters. Appendix E evaluates three exploration routes that reclaim that large-append regime.

The context sizes above are characters, not tokens. At the measured 3.9–4.2 characters per token on these texts, the 4.4 M-character row corresponds to ≈ 1.14 M tokens (minimum 1.08 M) and covers the million-token context windows now in deployment. A 1,050,000-token window is in deployment [22], and 10^6 -token windows ship across other frontier families [2].

Per-turn latency understates what a repair core delivers, so we also keep two throughput accounts, strictly apart. The *served* account credits a turn with all context bytes it delivers, the same crediting a cache gets. The *scanned* account counts only bytes physically retokenized, and for every full retokenizer the two coincide. Trace-weighted over the replay, one repair core serves 0.44 GB/s of context while scanning 3 MB/s, and its served curve crosses Gigatoken’s prewarmed ceiling at ~ 200 K context bytes (Figure 7c and Appendix D). At the 3 M-character shape, the served account reaches 1.4 GB/s for one core (Figure 7c). The served account is an incremental-repair account only. GPU full-tokenization throughput (§5.4) counts physically scanned bytes, and the two accountings are never mixed.

5.4 Full tokenization: latency and throughput

The complete GPU path sustains 3.8–4.7 GB/s across English, CJK, and templated corpora, and pre-tokenization alone reaches 30–77 GB/s. The strongest CPU configuration we could construct, fasttokens with one single-threaded worker per core and NUMA-local data placement, sustains 1.35–2.06 GB/s on the full 32-core host, and Gigatoken batch mode

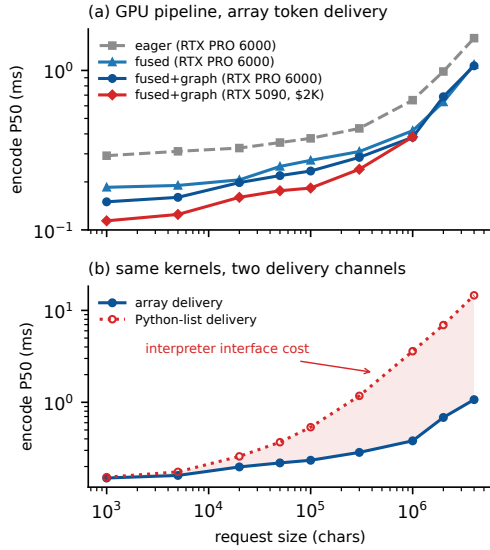


Figure 10. Full tokenization single-request encode P50 (Qwen3 family, steady state). (a) With array token delivery, every dispatch variant stays near or below one millisecond across three decades of request size, and the consumer RTX 5090 leads the server card. (b) The same fused+graph kernels behind two delivery channels. Materializing a Python list[int] adds interpreter interface cost that dominates beyond 100 K characters. Points beyond 1 M characters are extension points from the same protocol.

reaches 0.69–0.85 GB/s on the same host on an empty cache. The unpinned deployment reaches only 0.89–1.26 GB/s and degrades beyond 16 processes, so NUMA placement must be reported for CPU baselines, and we apply the same correction to every CPU number in this paper.

Session-initialization latency matters more to the service. Figure 10 shows steady-state single-request measurements. The CUDA-graph path with array output takes 0.15 ms at 1 K characters, 0.38 ms at 1 M, 0.68 ms at 2 M, and 1.07 ms at 4 M. Materializing a Python list[int] adds a host-side cost that dominates beyond 100 K characters. We report array and list delivery separately, because one measures the GPU pipeline and the other also measures the interpreter interface. Engine handoff through today’s Python APIs pays the list cost, and our TTFT results include it in full.

Table 3 uses the stricter serving protocol. Every request is a new real-text sample, encoded once. At 1 M characters, the GPU path answers in 0.87 ms with array output. That is 23.4× below the fastest previously published CPU configuration under the same protocol (8.2× at P90) and 6.9× below the strongest CPU number we measured from an empty cache, Gigatoken. At 2 M characters the respective values are 1.34 ms, 45.9 ms, and 8.78 ms. Python-list delivery clears every CPU row through 1 M characters (3.86 vs. 6.05 ms),

Table 3. Full tokenization single-request latency (ms) on identical real texts, one timed call per sample ($n=20$ per cell), context sizes in characters. The upper block is the normalized family (Qwen3, NFC), the last row the non-normalized family (Llama 3.1). Gigatoken uses a new object with an empty cache (construction excluded). The LoPT paper reports 116.8 ms on a 112-core node at LongBenchV2 lengths, an external anchor. The 28-process row is the same-host comparison (zero retries at every shape). The 4.4 M shape exceeds the graph path’s largest capture bucket (2^{22} bytes), so the GPU rows there run the same kernels without graph replay.

	P50 by context size				P90
	100 K	1 M	2 M	4.4 M	4.4 M
TokTIER GPU, array delivery	0.29	0.87	1.34	3.59	133.1
TokTIER GPU, Python list	0.56	3.86	8.61	19.87	144.5
Gigatoken, empty cache	0.64	6.05	8.78	19.42	22.3
fasttokens, 1 core	2.03	20.5	45.9	111.9	120.0
LoPT repro., 28 processes	7.91	42.2	95.3	198.5	228.4
HF serial, 1 core	24.8	360.3	762.9	1762.8	1833.5
TokTIER GPU, array (Llama 3.1)	0.27	0.80	1.31	3.28	4.1

then approaches Gigatoken at the largest sizes. Past 2 M the interpreter, not the GPU, binds that channel. Fresh samples also pay normalization quick-checks and buffer-geometry variance that a re-encoded corpus slice never sees, which is why this table’s 1 M median (0.87 ms) exceeds the steady-state 0.38 ms. Baseline comparisons use the sampled protocol only.

The largest Qwen3 samples expose a tail limitation. At 4.4 M characters, 4 of 20 inputs fail the GPU NFC quick check and pay CPU renormalization, lifting P90 to 133 ms. Llama 3.1 ships no normalizer and holds P90 at 4.1 ms on the same shape. We therefore report normalized and non-normalized families separately in Table 3, and full GPU normalization remains an implementation gap.

With 28 competing CPU processes, the eager pipeline’s P99 doubles from 0.52 to 1.05 ms at 50 K characters, while the graph path moves from 0.38 to 0.49 ms (Figure 17 in Appendix D). The kernels are unchanged.

5.5 Burst tails and capacity

Single-request medians do not show queueing behavior, so we replay 16 open-loop burst scenarios built from measured session shapes. A CPU-only front end needs seven times as many cores to keep continuation P99 flat, while initialization P99 stays between 210 and 490 ms at every tested core count. A four-core TokTIER repair pool plus one GPU keeps continuation P99 near 15 ms and initialization P99 between 6.9 and 46.6 ms (Figure 11). More CPU workers move the queueing knee without shortening one large full-context encode.

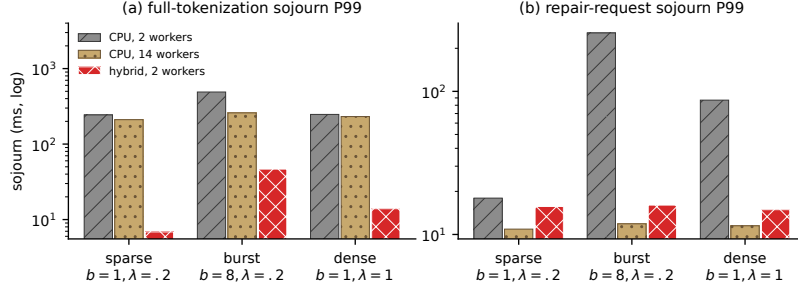


Figure 11. Tail behavior in recorded burst scenarios. Adding CPU workers does not remove the session-initialization service-time floor (a), and at low worker counts initialization work drags continuation P99 up (b). A four-core TOKTIER repair pool plus one GPU keeps continuation and initialization P99 low. The host-contention companion panel is Figure 17 in Appendix D.

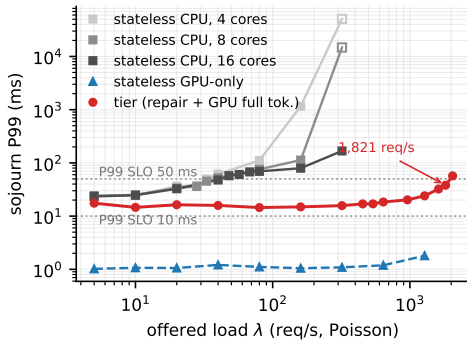


Figure 12. P99 sojourn time versus offered load (Poisson, measured mixture, 60 s steady state per point, and open markers are backlogged points reported as-is). The tier reaches 1,821 requests/s under a 50 ms P99 objective, while stateless CPU configurations saturate at 33–40 requests/s and only the GPU-only front end holds the 10 ms objective.

Figure 12 sweeps Poisson offered load over the measured request mixture. Session initializations account for 2.3% of requests and are sampled from the recorded initialization pool. A stateless CPU front end cannot meet a 10 ms P99 objective at any tested core count, because the $O(N)$ service floor alone exceeds the target. Under a 50 ms objective, 4, 8, and 16 cores sustain 33, 33, and 40 requests/s. The tier sustains 1,821 requests/s with four repair cores and one GPU, more than 45 $\times$ the 16-core stateless capacity. The 45 $\times$ ratio compares different hardware resources. It establishes the capacity of the tested configurations, and it does not isolate the benefit of session state from the benefit of the GPU, so we treat it as a system-capacity result rather than an equal-resource efficiency result. The bottleneck at 1,821 requests/s is the repair pool, not the GPU, whose full-tokenization P99 still reads 1.5 ms there.

A GPU-only stateless front end meets the 10 ms objective (P99 1.8 ms at 1,280 requests/s), but it ships the complete text of every request to the GPU, about 55 $\times$ the character volume

the tier sends. Heavy continuation appends of 30–100 K characters also hold incremental-repair service P99 at 13–17 ms independent of load, above the 10 ms target before queueing begins. The current router does not redirect these unusually large appends, and delta-size-aware routing would remove this tail case. Shadow verification at 5% sampling re-checked 9,161 requests across this sweep with zero mismatches, on an otherwise idle, preflight-gated machine.

5.6 vLLM in the loop

We submit either text or tier-produced token IDs to vLLM, paired on identical content with isolated KV prefixes. All runs in this section use the shipped Rust session store, a single implementation generation measured end to end. A prior-generation archive of the same regimes shows gains of the same sign and similar magnitude, because the store upgrade only accelerates the tier’s own component, which is 0.4–2% of TTFT. The gain comes from the engine side. Differencing the engine’s own metrics isolates a front-end segment that is neither queueing nor prefill, and the tier shrinks that segment from 37.6–356 ms per request to 9.8–48 ms while queue and prefill stay level (Figure 13a).

Figure 13b spans six regimes. At 28 K tokens with unloaded sequential streams, the two channels tie and the intervals cross zero, a null result we report deliberately, because single-stream benchmarks cannot see front-end tokenization. At 100 K-character contexts with 400-character appends and 15 requests/s Poisson arrivals, the tier reduces P50 TTFT by 26% ($n=675$). Replaying recorded burst timestamps at a similar mean rate improves P50 by 27% and P99 by 23% (2,961 to 2,288 ms), while P90 reverses by 28%. The reversal has two archived sources, GIL contention in the open-loop load generator and a more coherent burst shape reaching the scheduler, and the engine-measured end-to-end mean still improves. On gpt-oss-120B with 350 K-character contexts (95 K tokens), median TTFT falls by 32%, with the tier’s own component at 0.64 ms.

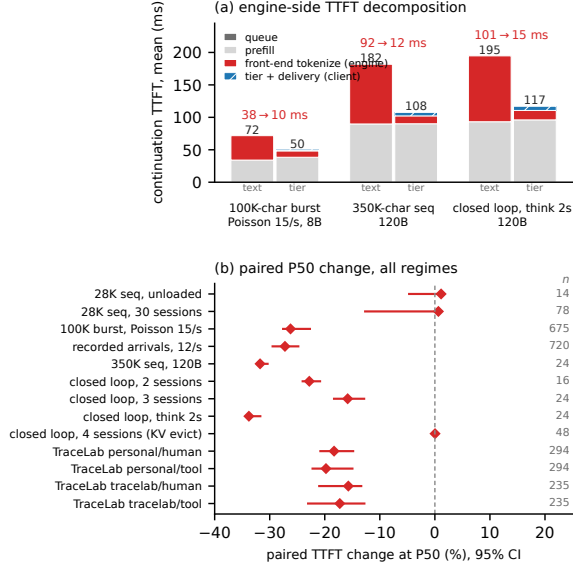


Figure 13. Session-continuation TTFT with vLLM in the loop, text-in vs. tier-in-front on identical paired content. (a) Engine-side decomposition from /metrics differencing shows the gap is the front-end tokenization segment, with queue and prefill unchanged, and the tier’s own client-side work below 6 ms. (b) Paired P50 change with bootstrap 95% CIs across all measured regimes, where the two-session point uses the control arm with a fresh KV cache. Under recorded arrivals P90 reverses (+28%) while P50 and P99 improve, a load-generator and batch-shape artifact whose decomposition is archived with the run.

Closed-loop experiments preserve causality within each session and run several sessions concurrently. At 95 K-tokens per session, median TTFT improves by 23% with two back-to-back sessions on a fresh KV cache, 16% with three, and 34% with two sessions and a two-second think time. At four sessions the engine cannot retain the needed KV state. Prefix reuse collapses, both channels degrade identically to 66-second full-prefill sojourns within $\pm 0.3\%$, and the tier’s own component stays below 1 ms. Tokenization placement is irrelevant past the engine’s KV capacity.

We also generate closed-loop sessions from TraceLab append and pacing distributions via the calibrated generator of §2.2. Across 12 paired runs, the median TTFT improvement is 16–20%, insensitive to pacing. Compaction steps land as 0.9–1.3 s full-prefill requests, while the tier’s state-rebuild component stays below 14 ms.

5.7 Cost and resource use

Applying the measured per-path costs to the 153,529 calls with complete token accounting gives 13.4 ms of CPU time per request for fasttokens and 2.3 ms for TokTIER repair over the same engine. Here r denotes the request completion rate

per inference GPU. At $r=0.5$ requests/s, those two values correspond to 6.7 and 1.1 front-end CPU cores per 1,000 GPUs. GPU full tokenization instead contributes 0.15 ms of GPU service time under this mixture. That is router GPU duty rather than CPU core time, and it normalizes to 0.1 router GPUs per 1,000 inference GPUs (Appendix D). We present this as a rate normalization rather than a cluster design. Topology, model scale, and batching enter only through r .

Direct power measurements show 2.06 GB/s at 221 W of CPU package power for the best 32-core fasttokens configuration, and 4.14 GB/s at 288 W of GPU board power plus 8 W of incremental host power. On the measured corpus the GPU path is about $1.5\times$ better in tokens per watt, under an accounting that favors the CPU, since package power excludes DRAM and PSU losses while the GPU figure is total board power. These figures support deployment feasibility, while the latency and state-reuse results remain the primary evidence.

Session state is also cheap to keep. Token IDs and byte spans account for about 16 bytes per token, a growing 500 K-token session holds its resident footprint under 15 MB in the shipped store, and the state is freed at session end (Figure 14a). The same figure prices the retention policy this state makes affordable. In the recorded traces, 5–7% of calls arrive more than five minutes after their predecessor and would miss a KV-lifetime cache, while a token-state TTL of one hour keeps 97–98% of calls on a session state hit and a day keeps 97–99% (Figure 14b). Holding hours of token state costs megabytes per session, where the corresponding KV state costs gigabytes.

5.8 Runtime verification

The shadow verifier earned its place by exposing a history-dependent bug in a widely deployed production Rust tokenizer.¹ During full-scale replay, 39 of 40 sampled sessions diverged from that engine while every fresh-process differential suite stayed green. A five-implementation adjudication established that the tier and the frozen reference agreed. The suspect engine returns different IDs for the same input after one earlier encode of a prefix at least 4,096 characters long, exactly the serving pattern of re-encoding a conversation after a small append. The wrong output has the same token count, so even a length check stays silent.

We also inject 434 faults into 2,999 session-shaped requests. The mutations cover ID substitution, deletion, duplication, adjacent swap, and a same-length tail resplit, the silent shape of the bug above. At 10% sampling, the verifier catches all 39 sampled faults, produces no false positives on 265 sampled clean requests, and drops no samples. Detection delay follows the expected geometric distribution. At 5% sampling, half of faulty campaigns are detected within 13 faulty requests and 90% within 42. The operator can therefore trade verifier

¹Reported upstream with a deterministic reproduction.

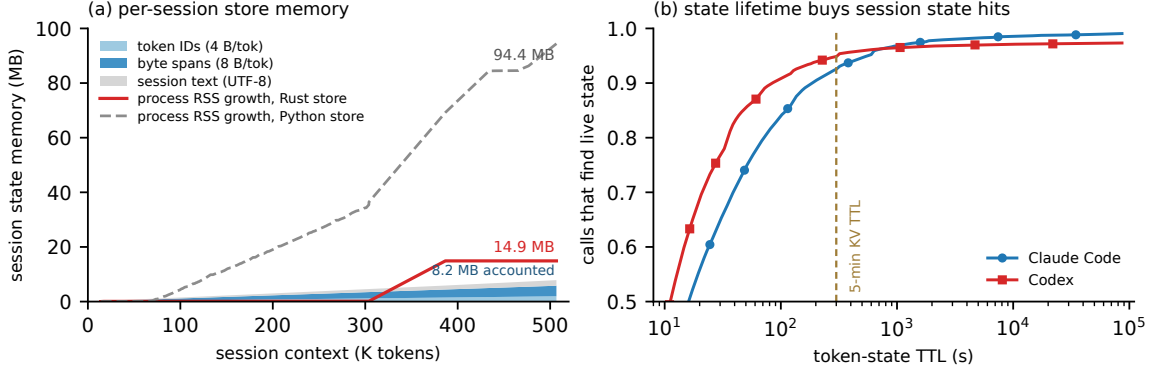


Figure 14. What session state costs and what its lifetime buys. (a) Store memory for one growing session on the replayed real trace. Accounted state is ~ 16 bytes per token, the Rust store’s resident growth stays between 1.2 and 22 MB per session across strains and families, and the prior Python store paid 70–129 MB for the same sessions. (b) Share of the 153,951 trace calls that would find live token state, as a function of state retention time. Raising retention from the 5-minute KV default to one hour converts 2–6% of calls from a session state miss to a session state hit.

cost for detection latency, while the drop counter exposes overload.

6 Related Work

CPU and segmented tokenization. HuggingFace tokenizers [9], tiktoken [21], Gigatoken [27], and vLLM’s proposed Rust front end [31] reduce the cost per input byte with Rust, SIMD, caching, and parallel CPU execution. They remain full-context methods from the point of view of an agent turn. Even a content-cache hit must identify cached pieces through the complete request. TOKTIER instead keeps the token sequence of a session and repairs only the changed region. The repair-window engine is pluggable, so faster CPU tokenizers can reduce its local cost once they pass the same correctness checks. Appendix D tabulates the design-space position.

LoPT [28] divides one request into overlapping chunks and matches results at chunk seams. Its goal is within-request CPU parallelism. TOKTIER uses a related overlap idea across time, matching a fresh window against a session’s cached sequence, and its acceptance condition is different. Our clean-room reproduction found inputs on which the length threshold stated in its paper admits a boundary outside the premise of its theorem. That result motivated the family-specific stable boundary check.

Incremental BPE [12] makes the BPE stage itself incremental. It maintains, for every prefix of a growing string, the state needed to extend the tokenization in $O(\log^2 t)$ per byte, and proves equivalence to the reference merge procedure when the vocabulary is canonical and its dependency graph admits a topological order. That line of work deliberately leaves the surrounding stages unchanged, and its authors note that normalization and regex pre-tokenization remain offline-oriented. TOKTIER works at the layer above. It keeps

the tokenizer implementation stateless, uses token and byte-span records as session state, and decides each splice with a per-request check defined over the full pipeline, so it admits families whose merge tables fall outside that premise and also supports mid-context edits.

A parallel line studies BPE as a streaming transduction. Mamouras et al. [16] bound the *delay* of a merge-rule list, the lookahead needed before a token can be finalized, and obtain a streaming algorithm whose memory is independent of the input length. The formal account of BPE by Berglund and van der Merwe [3] shows that a constant lookahead suffices in principle. These results describe the merge stage in isolation. The condition TOKTIER needs also covers the pre-tokenizer and the normalizer, which is why its boundary check is family-specific and is certified per request rather than assumed.

GPU tokenization. GPUDOK [13], BlockBPE [36], and cuDF subword tokenization [11] show that subword encoding can use GPU parallelism. Their supported specifications differ from the GPT-family reference tokenizers studied here. BlockBPE removes regex pre-tokenization, cuDF targets WordPiece, and GPUDOK checks a GPT-2-style byte-level pipeline against its own CPU implementation of that pipeline. None of them reports agreement with the frozen production tokenizers evaluated here. TOKTIER derives a parallel formulation of the reference rules and compares both final IDs and intermediate piece boundaries with frozen reference implementations, across six GPU-certified production configurations (Qwen3, Llama 3.1, gpt-oss, and the DeepSeek group of V3, V4-flash, and HY3) spanning four pattern families. To our knowledge, as of this writing it is the first GPU tokenizer shown byte-identical to its frozen production reference under this criterion.

Prefix-cached serving. vLLM [14], RadixAttention [38], Mooncake [25], DistServe [39], Splitwise [23], CacheGen [15], Dynamo [19], and continuous batching [37] optimize model execution after token IDs exist, and provider prompt caching exposes the same reuse to users [1]. TOKTIER addresses the preceding front-end step. Reuse has begun to appear at that step as well. Serving front ends now ship in-memory tokenizer prefix caches that map a text prefix to the token IDs it produced, keyed by a digest of that prefix [20, 29]. These caches split a request only at special-token boundaries, where the pieces compose by construction, so an append that follows ordinary text yields no reuse. They also live inside a single front-end process, and they discard the prefix text once it is hashed, so a hit is trusted rather than re-checked. TOKTIER instead keeps token IDs together with their source byte spans as session state, matches that state against the request text, and accepts a splice wherever the per-request boundary check of §3.2 passes. It uses token-ID request interfaces that serving stacks already expose, such as vLLM’s `prompt_token_ids` (§4.5) and Dynamo’s pre-tokenized backend requests, so it deploys without changing the model scheduler.

Agent workloads. Public inference traces [25, 32] report request sizes and arrival rates but generally not the relation between a session’s previous context and its new text. TraceLab [40] and CacheWise [30] characterize coding-agent sessions for KV-cache management, and their measurements independently support the long-context, small-append, high-hit-rate pattern used in this paper. TraceLab discards text, so it cannot replay tokenization. Our traces retain counts only for privacy, while the public autonomous-agent corpus [10] releases message text at preserved lengths, with filler substituted for redacted spans. CPU interference studies [4] corroborate the contention regime of §5.5.

Differential validation. Differential testing has a long history in compilers [34], and translation validation checks a produced result against a specification without proving the entire implementation [17, 24]. TOKTIER applies this division to tokenization. A theorem justifies an accepted repair splice in a family-level model, version-pinned differential campaigns check the frozen implementation, and runtime sampling covers history-dependent failures. The paper does not claim formal verification of the complete tokenizer stack (Appendix D).

7 Limitations

We collect the system’s limitations here, from the physical floor of the GPU path to the boundary of what zero divergence establishes.

What limits the GPU path. The GPU tokenizer uses less than 80 GB/s of DRAM traffic, about 4% of the card’s bandwidth, at low SM occupancy. Raising the RTX PRO 6000

power limit from 450 to 600 W changes no result. The limiting resource is the dependency chain of exact BPE merges. Each merge is a dependent lookup whose result decides the next, so the floor is chain depth times memory round-trip, a latency roofline.

Three measurements place the implementation at that floor. First, measurements across five GPUs spanning three architecture generations follow sustained clock more closely than SM count, bandwidth, or price, and the \$2,000 consumer RTX 5090 runs 11–17% faster than the server-class card (Appendix D). Second, optimizations that pay on throughput-bound kernels measurably do not pay here. L2 residency pinning loses 11–12%, per-architecture launch retuning moves results within $\pm 1\%$, and an on-GPU piece-memoization prototype with a 96.8% hit rate returns +0.8%. Third, a gated prototype that commits rank plateaus in parallel gains 42% CJK throughput but diverges from the reference on 4.0% of Qwen pieces and 0.34% of Llama pieces. We do not use it. The remaining chain is the price of exactness, and it implies a router-resident tier wants one cheap high-clock card rather than a datacenter GPU.

Coverage. Incremental repair requires a supported family and a stable boundary inside the matched region. Two of the 17 families we examined do not satisfy the current predicate and always use full tokenization. WordPiece follows a different boundary argument and remains on the CPU path in the current system. Added-token extraction and one NFC normalizer also stay on the CPU. These fallbacks preserve output correctness and shrink the fraction of traffic that receives the fast path.

Session-state construction. The GPU encoder currently emits token IDs without source byte spans. A request that initializes session state therefore runs the reference tokenizer to obtain spans. In the measured engine-in-loop runs this adds at most 14 ms to a request whose model prefill takes 0.9–1.3 s. Exporting spans on the GPU is the highest-leverage missing piece of engineering. Cross-worker state transfer is the other planned extension. Today a request that loses worker affinity becomes a session state miss and stays correct (§3.4).

Workload scope. The personal trace panel covers six users and nine machines and is concentrated on coding agents. Provider aggregates, a public autonomous-agent trace, and TraceLab reduce the risk that one client or group determines the result, but they do not cover all agent applications. The benefit also depends on context length, since a fully pre-warmed content cache wins on quiet cores at 100 K characters and below (§5.3) and TOKTIER grows stronger as the context grows relative to the append.

Validation boundary. Zero divergence in the reported campaigns is evidence for the tested artifacts, not a proof

about future tokenizer versions. Each new snapshot must repeat family admission and differential validation. The shadow verifier stays deployed, because one observed failure depends on prior request history and cannot be ruled out by fresh-process testing alone. All personal traces were collected with consent under a count-only discipline (Appendix B), and the history-dependent-tokenizer bug was reported upstream with a deterministic reproduction before publication.

8 Conclusion

Agentic serving repeatedly submits long contexts after small updates. KV prefix caching exploits that structure only after the front end has produced token IDs, and the tokenizer caches that front ends ship today reuse earlier work only inside one process and only up to the last special-token boundary. TOKTIER carries the same stateful view into tokenization. It repairs continuing sessions at a checked boundary, sends large session initializations through an exact GPU path, and falls back to the reference implementation whenever a fast-path condition is unavailable. The result is a front end whose common-case work follows the change in the request, with output compatible with existing models and prefix caches.

Deployment. Our evaluation places one GPU at the serving entry point and uses session affinity across repair workers. The tokenizer consumes little GPU bandwidth, so colocation with an inference GPU is possible, and the cross-generation results suggest one high-clock consumer card suffices (§7). A production deployment would also need state replication, admission control, and a state-lifetime policy, none of which changes the tokenization algorithms, though each can affect the session state hit rate.

Delta-aware routing. Continuation appends beyond ~30 K characters hold incremental-repair service P99 above a 10 ms objective (§5.5), and the shipped router does not redirect them. Head-to-head measurements show where each path is optimal. Repair cost follows the append and is flat in context, while a GPU rebuild of the complete context costs 1.1–3.2 ms up to window-max scale, so rerouting appends of 50 K characters and above would cut their service time by 6–28× relative to the shipped repair path. Figure 18 in Appendix E maps the measured routing regions, and the three explored routes are detailed there. Only 0.8% of measured appends are that large, and these numbers use exploration-prototype accounting, so the shipped default remains repair for every session continuation until the rerouted path passes the same certification battery.

References

- [1] Anthropic. 2024. Prompt Caching with Claude. Explicit 5-minute and 1-hour cache TTLs.
- [2] Anthropic. 2026. Models overview. <https://platform.claude.com/docs/en/about-claude/models/overview>. Claude Fable 5 and Claude Opus 5: 1M token context window. Accessed July 30, 2026.
- [3] Martin Berglund and Brink van der Merwe. 2023. Formalizing BPE Tokenization. arXiv:2309.08715.
- [4] Euijun Chung, Yuxiao Jia, Aaron Jezghani, and Hyesoon Kim. 2026. Characterizing CPU-Induced Slowdowns in Multi-GPU LLM Inference. arXiv:2603.22774.
- [5] DeepSeek-AI. 2024. DeepSeek-V3 Technical Report. arXiv:2412.19437.
- [6] DeepSeek-AI. 2024. DeepSeek-V3 tokenizer.json. HuggingFace model artifact, revision e815299b. <https://huggingface.co/deepseek-ai/DeepSeek-V3>.
- [7] Xiang Deng, Jeff Da, Edwin Pan, et al. 2025. SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks? arXiv:2509.16941.
- [8] fasttokens contributors. 2026. fasttokens: A Fast BPE Tokenizer with a Rust Backend. Version 0.2.0, Apache-2.0. Supported by vLLM v0.23.0 and later as an opt-in tokenizer backend that must be installed separately. <https://github.com/crusoecloud/fasttokens>.
- [9] HuggingFace. 2019. HuggingFace Tokenizers. Rust library; version 0.22.2 in the frozen environment of this paper. <https://github.com/huggingface/tokenizers>.
- [10] Inferact. 2026. codex_swebenchpro_traces: Agentic Workload Traces of Codex on SWE-Bench Pro. HuggingFace dataset, MIT license. Redacted spans are replaced by length-preserving filler text. File codex_swebenchpro.json, SHA-256 prefix 670f1ae8325fd70a, retrieved July 13, 2026.
- [11] Vibhu Jawa. 2021. Run State of the Art NLP Workloads at Scale with RAPIDS, HuggingFace, and Dask. NVIDIA Developer Blog; BERT WordPiece batch tokenization. Accessed July 31, 2026. <https://developer.nvidia.com/blog/run-state-of-the-art-nlp-workloads-at-scale-with-rapids-huggingface-and-dask/>.
- [12] Shenghu Jiang and Ruihao Gong. 2026. Incremental BPE Tokenization. In *Proceedings of ICML*. arXiv:2605.30813.
- [13] Venu Gopal Kadamba and Kanishkha Jaisankar. 2026. GPOTOK: GPU Accelerated Byte Level BPE Tokenization. arXiv:2603.02597.
- [14] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of SOSP*.
- [15] Yuhao Liu, Hanchen Li, Yihua Cheng, et al. 2024. CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving. In *Proceedings of SIGCOMM*.
- [16] Konstantinos Mamouras, Angela W. Li, and Yudi Yang. 2026. An Efficient Algorithm for Streaming BPE Tokenization. *Proceedings of the ACM on Programming Languages* 10, PLDI, Article 252 (2026), 2085–2108 pages. <https://doi.org/10.1145/3808330>
- [17] George C. Necula. 2000. Translation Validation for an Optimizing Compiler. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*.
- [18] NVIDIA. 2024. NVIDIA Blackwell Platform: GB200 NVL72. Vendor-reported claim: up to 30× LLM-inference throughput vs. the same number of H100 GPUs; named mechanisms are FP4 in a second-generation Transformer Engine and fifth-generation NVLink. Accessed July 31, 2026. <https://nvidianews.nvidia.com/news/nvidia-blackwell-platform-arrives-to-power-a-new-era-of-computing>.
- [19] NVIDIA. 2025. NVIDIA Dynamo: A Datacenter-Scale Distributed Inference Serving Framework. Router documentation: backend handlers receive pre-tokenized requests.
- [20] NVIDIA. 2026. dynamo-tokenizers: Tokenizer Backends and Prefix Cache for the NVIDIA Dynamo Front End. Rust crate. In-memory prefix cache keyed by a digest of the text prefix and split at special-token boundaries; enabled by default since June 2026. <https://github.com/ai-dynamo/frontend-crates>.
- [21] OpenAI. 2023. tiktoken: A Fast BPE Tokeniser for Use with OpenAI’s Models. <https://github.com/openai/tiktoken>.

- [22] OpenAI. 2026. GPT-5.6 Sol Model | OpenAI API. <https://developers.openai.com/api/docs/models/gpt-5.6-sol>. Context window: 1,050,000 tokens. Accessed July 30, 2026.
- [23] Pratyush Patel, Esha Choukse, Chaojie Zhang, et al. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *Proceedings of ISCA*. Azure LLM inference traces.
- [24] Amir Pnueli, Michael Siegel, and Eli Singerman. 1998. Translation Validation. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*.
- [25] Ruoyu Qin, Zheming Li, Weiran He, Jiale Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading More Storage for Less Computation — A KVCache-Centric Architecture for Serving LLM Chatbot. In *Proceedings of the 23rd USENIX Conference on File and Storage Technologies (FAST)*. USENIX Association, 155–170.
- [26] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language Models are Unsupervised Multitask Learners. GPT-2; byte-level BPE with regex pre-tokenization. <https://github.com/openai/gpt-2>.
- [27] Marcel Rød. 2026. Gigatoken: SIMD and Cache Hierarchies for 1000x Faster Byte-Pair Encoding Tokenization on Modern CPUs. Audited at version 0.9.0, commit 0d9765fa. <https://github.com/marcelroed/gigatoken>.
- [28] Wei Shao, Lingchao Zheng, Pengyu Wang, Peizhen Zheng, Jun Li, and Yuwei Fan. 2026. LoPT: Lossless Parallel Tokenization Acceleration for Long Context Inference of Large Language Model. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, San Diego, California, United States, 33107–33122. <https://doi.org/10.18653/v1/2026.acl-long.1529>
- [29] The llm-tokenizer authors. 2026. llm-tokenizer: A Caching Tokenizer Layer for LLM Gateways. Rust crate, first released January 2026. In-memory caching of tokenization results. <https://github.com/lightseekorg/smg>.
- [30] Shubham Tiwari, Tapan Chugh, Nash Rickert, Simon Peter, Ratul Mahajan, and Haiying Shen. 2026. CacheWise: Understanding Workloads and Optimizing KVCache Management for Efficiently Serving LLM Coding Agents. arXiv:2606.16824.
- [31] vLLM contributors. 2026. [RFC]: Rust front-end. GitHub issue 40846, open; proposed as an experimental preview behind an opt-in flag, with the Python front end kept as the default. <https://github.com/vllm-project/vllm/issues/40846>; parity tracking #44280.
- [32] Yuxin Wang, Yuhang Chen, Zeyu Li, Xueze Kang, Yuchu Fang, Yeju Zhou, Yang Zheng, Zhenheng Tang, Xin He, Rui Guo, Xin Wang, Qiang Wang, Amelie Chi Zhou, and Xiaowen Chu. 2025. BurstGPT: A Real-World Workload Dataset to Optimize LLM Serving Systems. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*. 5831–5841. <https://doi.org/10.1145/3711896.3737413>
- [33] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Proceedings of NeurIPS*. arXiv:2405.15793.
- [34] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and Understanding Bugs in C Compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*.
- [35] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *Proceedings of ICLR*. arXiv:2210.03629.
- [36] Amos You. 2025. BlockBPE: Parallel BPE Tokenization. arXiv:2507.11941; ES-FoMo III workshop at ICML 2025.
- [37] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 521–538.
- [38] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, et al. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Proceedings of NeurIPS*. RadixAttention prefix caching.
- [39] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-Optimized Large Language Model Serving. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 193–210.
- [40] Kan Zhu, Mathew Jacob, Chenxi Ma, Yi Pan, Stephanie Wang, Arvind Krishnamurthy, and Baris Kasikci. 2026. TraceLab: Characterizing Coding Agent Workloads for LLM Serving. arXiv:2606.30560; dataset release v0.0.1, CC BY 4.0, syfi_coding_trace SHA-256 prefix 9d265eae69a31cae. <https://github.com/uw-syfi/TraceLab>.

A The Splice Theorem

This appendix states and proves the losslessness theorem behind the incremental repair’s splice certificate (§3.2). Full per-family discharge proofs, the $v1 \rightarrow v2$ repair history (the digit-grouping counterexample of §3.2), and the adversarial discovery battery are in the companion document shipped with the artifact. The per-family results are summarized in §A.3.

A.1 Setting

Definition A.1 (Token records). For a string S , the no-post-processing tokenization is an ordered sequence $F(S) = (e_1, \dots, e_n)$ of records $e_i = (\text{id}_i, a_i, b_i)$ with $[a_i, b_i] \subseteq [0, |S|)$ the source span reported by the offset mapping. Spans may leave gaps (dropped whitespace), may repeat (several byte-level tokens of one multi-byte character), and never partition $[0, |S|)$ in general. Sequence *index* is the only structure the theory uses.

Definition A.2 (Front end and units). Write the pipeline as $F = E^* \circ G$:

- G (*front end*) maps S to an ordered sequence of *units* $u_1, \dots, u_m$, each $u = (\text{kind}, \text{payload}, I)$ with $\text{kind} \in \{\text{piece}, \text{added literal}\}$, payload the normalized text the model stage will consume, and $I \subseteq [0, |S|)$ the source interval. Source intervals are ordered and non-overlapping but need not cover S . For HF fast tokenizers $G = \text{added-token extraction}$, then per-segment normalization, then pre-tokenization.
- E (*model stage*) maps one unit to a *token-ID* sequence, as a function of $(\text{kind}, \text{payload})$ alone: an added literal maps to its single ID; a piece maps to the IDs of its WordPiece/BPE encoding. E^* concatenates:

$$E^*(u_1, \dots, u_m) = E(u_1) \parallel \dots \parallel E(u_m).$$

The losslessness claim is on the ID sequence

$$F_{\text{id}}(S) = (\text{id}_1, \dots, \text{id}_n).$$

The global spans a_i, b_i of Definition A.1 are *not* outputs of E (a payload repeated at two positions has one ID but two spans); the merge reconstructs each token’s span from its unit’s source interval I plus the chunk base, an operation that never crosses a junction and is therefore outside the homomorphism. Writing F for the ID sequence below is thus without loss.

Assumption 1 (Protocol). *Chunks are encoded with `add_special_tokens=False`, no padding, no truncation; sequence-level post-processing (special-token wrapping) is applied once, after the merge. This matches the deployed merge path.*

Assumption 2 (Pipeline fidelity). *F factors as in Definition A.2 with E per-unit deterministic and stateless across units, for `tokenizers==0.22.2` as pinned by the manifest. BPE dropout is off. This is the load-bearing assumption: the model stage is per-piece, with no cross-piece state. Our differential campaigns support it for the reference implementation; the history-dependent divergence of §5.8 is precisely a violation of its analogue by a non-reference engine.*

Lemma A.3 (Concatenation homomorphism). *For unit sequences U, V : $E^*(U \parallel V) = E^*(U) \parallel E^*(V)$.*

Proof. Immediate from the definition of E^* and Assumption 2. $\square$

Table 4 records the checked front-end configuration of the six tokenizers analyzed in this appendix. This set includes the two WordPiece families and is distinct from the six GPU-certified configurations of Table 1. Every locality statement is made *per configuration*, not for “HF tokenizers” at large.

A.2 The splice certificate and the main theorem

Definition A.4 (Splice certificate). Let S be covered by chunk windows $S_i = S[x_i : y_i]$, $i = 1..N$, with $x_1 = 0$, $y_N = |S|$, and overlaps $x_{i+1} < y_i$. A *splice certificate* is a sequence of junction positions $0 = b_0 < b_1 < \dots < b_N = |S|$ with $b_i \in [x_{i+1}, y_i]$ for $0 < i < N$, such that:

- (C1) no unit of $G(S)$ has a source interval straddling any b_i ; write $G(S) = A_1 \parallel \dots \parallel A_N$ where A_i collects the units with source interval inside $[b_{i-1}, b_i]$;
- (C2) for each i , the units of $G(S_i)$ with source interval inside $[b_{i-1}, b_i]$ (in global coordinates) are *exactly* A_i (same kinds, payloads, and intervals), and no unit of $G(S_i)$ straddles b_{i-1} or b_i .

Theorem A.5 (Losslessness under a splice certificate). *Under Assumptions 1–2, if a splice certificate exists, then concatenating, for $i = 1..N$, chunk i ’s records at the token indices produced by its middle block A_i yields exactly $F(S)$.*

Proof. By (C1) and Lemma A.3,

$$\begin{aligned} F(S) &= E^*(G(S)) = E^*(A_1 \parallel \dots \parallel A_N) \\ &= E^*(A_1) \parallel \dots \parallel E^*(A_N). \end{aligned}$$

Fix i and write $G(S_i) = L_i \parallel A_i \parallel R_i$, which is exactly what (C2) states: L_i (resp. R_i) are the units of $G(S_i)$ left of b_{i-1} (right of b_i), possibly corrupted by the window edges; the middle block equals A_i verbatim. By Lemma A.3 chunk i emits $E^*(L_i) \parallel E^*(A_i) \parallel E^*(R_i)$, so its records at indices

$$[|E^*(L_i)|, |E^*(L_i)| + |E^*(A_i)|]$$

are $E^*(A_i)$: E depends only on kind and payload (Assumption 2), which (C2) pins. Concatenation over i gives $F(S)$. $\square$

Remark 1. The merge is defined by token *index*. Nothing in Theorem A.5 mentions spans, so duplicate spans, gaps, and multi-byte grouping are inert. The N -chunk case needs no induction: the unit-block factorization handles all junctions at once.

Corollary A.6 (Splice shifting through an equal run). *The deployed merge splices at the end c_i of a matched run, not at the certified boundary b_i inside it. Because $[b_i, c_i]$ lies within the equal (span, id) run that produced the certificate, the two chunks’ record sequences (write L and R for them) coincide there ($L[b_i:c_i] = R[b_i:c_i]$), so*

$$\begin{aligned} L[:c_i] \parallel R[c_i:] &= L[:b_i] \parallel L[b_i:c_i] \parallel R[c_i:] \\ &= L[:b_i] \parallel R[b_i:c_i] \parallel R[c_i:] \\ &= L[:b_i] \parallel R[b_i:], \end{aligned}$$

splicing at c_i equals splicing at b_i . Under the standard overlap geometry the actual cuts are monotone and non-crossing, so Theorem A.5 certifies at $\{b_i\}$ while the implementation runs at $\{c_i\}$.

Remark 2 (Cost of the abstraction). All difficulty lives in discharging (C1)/(C2). Three routes: (a) *global front-end pass*: run G once, serially, and hand whole units to workers (the certificate holds by construction, at an Amdahl cost equal to the front-end share of runtime); (b) *local certification* at synchronizing boundaries: check a bounded window around a candidate b_i ; (c) *matched-run discovery* (the incremental-repair protocol): search for equal token runs, then obtain a certificate from them, sound for WordPiece via a continuation-witness certificate, and for byte-BPE only together with an explicit synchronizing-boundary check (§A.3).

A.3 Discharging the certificate per family (summary)

The companion document discharges (C1)/(C2) per checked front-end configuration. For *BERT WordPiece*, boundary discovery from a matched run is sound with a continuation witness (the deployed certificate). A matched run containing a non-continuation boundary pins the pre-tokenizer’s state. For the *byte-BPE families*, a matched ID run alone is *not* a certificate. Context-dependent digit grouping defeats it (the counterexample of §3.2). The deployed check therefore requires the run to contain a character-class transition from a per-family *synchronizing set*, proven to reset

Tokenizer	Normalizer	Pre-tokenizer	Added flags	max lit.
Qwen3-8B	NFC	regex (\p{N} single) + ByteLevel	none	20
Llama-3.1-8B	none	regex (\p{N}{1,3}) + ByteLevel	none	30
DeepSeek-V3/V4	identity	3×Split + ByteLevel	none	23
gpt-oss-120b	none	regex (case-aware) + ByteLevel	none	19
BERT cased	clean_text, CJK	BertPreTokenizer	none	6
BERT uncased	+ lowercase, accents	BertPreTokenizer	none	6

Table 4. Front ends of the six tokenizers analyzed in this appendix. “max lit.” = length in characters of the longest added-token literal, the bound the leftmost-longest splitter needs. “none” = no `lstrip/rstrip/single_word` added tokens. The unbounded-absorption attack is thereby out of scope *by checked configuration*, not by argument. DeepSeek-V3 and V4 ship byte-identical normalizers, pre-tokenizers, and merge tables, verified by content hash on the frozen snapshots. They differ only in their added-token lists, so this row (and every base-tokenizer result in the paper) covers both versions.

the pre-tokenizer under every left context. The sets are derived per pattern (single-digit vs. {1,3}-digit grouping, case-aware boundaries for o200k). After the Unicode-version-skew episode of §5.2, every character-class table they consult is probed directly out of the reference engine rather than any standard library. DeepSeek’s three-splitter sequential composition reduces to the same per-character predicate. Acceptance on natural text is preserved. Re-replaying both trace corpora under the certificate configuration accepts every splice the length-only check accepts (§5.2).

B Workload Characterization: Details

This appendix carries the full workload characterization (Fig. 15) and the collection and parsing detail behind §2. Nothing here is needed to follow the paper’s argument. All of it is needed to reproduce or audit the workload numbers.

B.1 Collection methodology and privacy discipline (L1)

The L1 collector is a single-file, zero-dependency parser of the agents’ local session logs. It exports *counts only*. The fields are token usage as reported by the serving API (input, cache-read, cache-write where available, output), character counts of new context (text is length-counted in memory and discarded), timestamps, model names, and client versions. Session and project identifiers are HMAC-hashed with a key that never leaves the contributor’s machine. A mechanical self-audit (field whitelist plus string-shape validation) refuses to package anything else. Contributors authorize collection and inspect the human-readable output before it leaves their machine. One contributor’s Codex traffic is agent-initiated (their Claude Code sessions drive Codex), so its pacing inherits the human loop while its per-call anatomy sits between our interactive and autonomous sources.

B.2 Parsing hazards

Three parsing hazards are worth recording for reproducers. First, both ecosystems’ logs report one API call as multiple streamed records, which must be deduplicated (22,274→9,987

and 62,090→40,741 records-to-calls on one machine’s data, respectively). Second, one ecosystem’s system-prompt text never appears in its logs, so character-based increment estimates for first calls are lower bounds. We therefore report token-based figures where provenance matters. The third is subtler. One ecosystem’s fork/resume machinery replays the parent session’s history (usage-bookkeeping events included) into the new session’s log file, so a naive cross-file parse double-counts replayed calls. We detect these replayed prefixes (≥ 6 consecutive calls with identical usage four-tuples, corroborated by degenerate timestamps or a fork-point context of ≥ 40 K tokens) and drop them. The drop removes 26,578 bookkeeping copies, 14.7% of parsed calls. A $\approx 0.8\%$ residue of single-record cross-file duplicates in the other ecosystem’s logs survives this rule and is disclosed rather than removed.

B.3 The public autonomous-agent trace (L3)

In `codex_swebenchpro`, the release preserves conversation structure and message lengths, and substitutes a repeating length-preserving filler passage for the spans it redacts. Every assistant message is filler. Human and tool messages keep their prompts, commands, file paths, and captured program output verbatim, and carry filler in place of the redacted part of their bodies. Most corpus characters are verbatim text. We disclose this property wherever it matters to a measurement. Its aggregate cache hit rate (94.2%) is the figure its publishers report for this corpus, collected on a workload disjoint from ours, making it an independent anchor rather than a re-measurement of our own traces. Its heavier per-call increments (median 3.8 K characters) reflect that tool output is chunkier when no human is pruning it. The SWE-smith trajectory corpus contributed by the replay campaigns of §5.2 comprises 761 K requests across 26.1 K trajectories, of which the certificate-replay campaigns exercise a 200-stream sample.

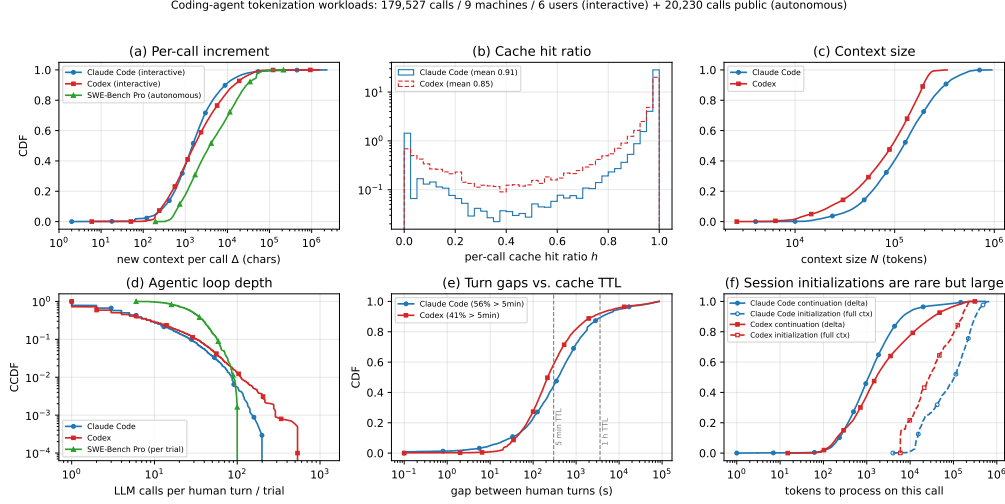


Figure 15. Anatomy of coding-agent tokenization workloads across nine machines, two ecosystems, and a public autonomous-agent trace. (a) Per-call increment Δ . (b) Per-call cache hit ratio h . (c) Context size N . (d) LLM calls per human turn. (e) Turn-gap distribution against the 5-minute and 1-hour cache TTLs. (f) Tokens-to-process per call, session continuations versus session initializations. Continuations process their small delta while initializations carry the full context, two orders of magnitude apart. The joint view of the (c) and (f) token accounting is Fig. 2 in the main text.

B.4 TraceLab aggregates (L4)

Our parse of the released TraceLab [40] v0.0.1 trace reproduces its full aggregates. They are 4,265 sessions from 43 developers, 357,161 LLM steps, 432,510 tool calls, 54.90 B input tokens (52.56 B cached prefix + 2.34 B append), and 186.9 M output tokens. Its median step carries 126,180 cached-prefix tokens against 857 appended for Claude Code (115,584/886 for Codex). The token-weighted cache hit rate is 95.7% overall and 84.4% on user-triggered steps. Prefix-cache reads account for 59.5% of dollar spend. The per-step retokenization amplification $(P+A)/A$ quoted in §2.3, computed by our script over that release, splits as median 155 $\times$ for Claude Code and 119 $\times$ for Codex, with a pooled P90 of 768 $\times$. Because TraceLab discards text, we use it two ways only. It serves as an external check on L1–L3, and as a calibration profile for the load generator driving the closed-loop experiments of §5.6.

B.5 Temporal detail

TraceLab reports provider-side cache behavior consistent with our client-side turn gaps. Eviction begins at pauses above 5 minutes, and pauses above an hour almost always miss [40]. Computed from its released trace, the mean cached share of a step falls from 0.96 (preceding gap under 1 minute) and 0.93 (1–5 minutes) to 0.70 (5–15 minutes), 0.58 (15–60 minutes), and 0.17 beyond an hour, the per-bucket decay behind the summary in §2.5.

B.6 Note on the GPU efficiency trend

The 30 $\times$ headline for generational inference throughput (Hopper to Blackwell NVL72) is a vendor-reported comparison against the same number of H100 GPUs [18]. The mechanisms that source names are FP4 in a second-generation Transformer Engine and fifth-generation NVLink across the NVL72 domain. The scissor argument of §2.6 discounts the headline to 5–15 $\times$ and needs only its sign.

C Sweeping the Append Size

Table 2 varies the context while drawing append sizes from the measured workload (median 1.5–1.6 K characters per call), so it shows how each method scales with N but not with Δ . This appendix sweeps the second axis. It uses fixed append sizes of 1 K, 5 K, 10 K, 50 K, and 100 K characters at every context shape of Table 2, all four methods on identical texts (Table 5). For calibration, on the trace pool behind these experiments the per-call increment has median 1.4 K characters, P90 6.9 K, and P99 38 K. The two largest settings are past the 99th percentile (0.8% and 0.5% of measured calls reach 50 K and 100 K), so they price a stress regime, not the typical loop.

Protocol. The context texts are the frozen session prefixes of Table 2. The deltas are fresh fixed-length slices of the same transcript stream, taken in order from each session’s append point, with the slice positions recorded in the archive. Each cell has $n=24$ samples (4 sessions $\times$ 6 slices), one timed call per sample per process, on one pinned core. Each repair sample uses a fresh session. We bootstrap on the prefix, run

Table 5. Append-size sweep. P50 latency in ms per append ($n=24$ per cell), at every context shape of Table 2. Context sizes (ctx) and append sizes (Δ) are in characters. Same host, tokenizer family (Qwen3), and timing discipline as Table 2. The four methods time byte-identical texts. Deltas are fixed-length contiguous slices of the same real transcript stream, cut in order from each session’s append point (controlled length, real text), unlike the main table’s natural per-call sizes. Gigatoken runs in its most favorable mode, cache prewarmed on the session text. Every repair cell took the standard repair path, with zero retries, no fallback, and 140/140 output spot-checks identical to the reference tokenizer.

ctx \ Δ	1 K	5 K	10 K	50 K	100 K
<i>repair (ours)</i>					
100 K	0.43	1.50	2.66	12.2	25.8
500 K	0.76	2.28	4.01	17.2	30.2
1 M	1.29	2.73	4.41	17.0	32.2
2 M	1.67	3.18	4.71	18.2	32.7
3 M	2.14	3.71	5.80	18.6	33.6
4.4 M	3.52	4.89	6.67	19.7	34.7
8 M	6.17	8.22	10.2	24.3	40.1
<i>Gigatoken, prewarmed cache</i>					
100 K	0.13	0.15	0.18	0.32	0.61
500 K	1.18	1.19	1.21	1.38	1.54
1 M	2.52	2.54	2.46	2.75	2.86
2 M	4.78	4.79	4.84	5.05	5.25
3 M	7.34	7.37	7.42	7.51	7.64
4.4 M	11.7	11.6	11.6	11.8	12.1
8 M	23.2	23.2	22.8	23.3	23.6
<i>fastokens, full retokenization</i>					
100 K	0.73	0.81	0.90	1.68	2.73
500 K	4.28	4.32	4.50	5.22	6.13
1 M	9.58	8.98	9.19	10.2	10.9
2 M	21.2	17.8	17.8	18.9	20.2
3 M	31.9	28.0	26.6	27.9	28.9
4.4 M	55.4	48.9	45.1	45.8	47.9
8 M	104.7	100.2	93.5	87.5	86.6
<i>HF serial, full retokenization</i>					
100 K	22.2	23.0	24.2	36.4	52.8
500 K	164.5	163.6	172.2	178.1	195.1
1 M	322.1	321.6	323.1	337.5	352.5
2 M	665.7	659.3	662.4	673.2	694.4
3 M	986.0	983.6	982.2	996.5	1008.4
4.4 M	1545.4	1537.0	1537.0	1543.0	1561.0
8 M	3156.5	3140.1	3150.4	3170.9	3162.8

one untimed append of the session’s first natural delta (so the timed call measures the steady-state repair path rather than the one-time first-append bookkeeping visible in the 8 M P90 of §5.3), and then time the append of one fixed-size slice. Gigatoken gets a fresh tokenizer object per cell, prewarmed on the same session text outside the timing region. The serial engines retokenize the same full text. Under this protocol the 1 K column reproduces Table 2 on its nearby shapes. The three baselines land within $\pm 7\%$ of the archived medians, and repair lands between -16% and $+7\%$ (the controlled 1.0 K

append is smaller than the natural 1.5 K median, and the incremental-step protocol excludes the first-append cost that the main table’s mix includes).

Reading. Repair’s cost separates into the two terms its bound predicts. Down a column sits the $O(N)$ term. At $\Delta=1$ K the P50 grows from 0.43 to 6.17 ms as the context grows 80 $\times$, the prefix-verification scan. Across a row sits the $O(\Delta)$ term. Moving from 1 K to 100 K adds 25.4 to 33.9 ms depending on shape. That is the appended bytes tokenized at 2.9–3.9 MB/s, the same regime as the scanned account of Fig. 16. The two terms compose near-additively. Interior cells sit within 17% of the line through their row’s endpoints (median deviation 7%). The full retokenizers move only through the total length $N+\Delta$. At the 100 K shape, a 100 K append doubles the text and HF’s time rises 2.4 $\times$. At 1 M and above, the same append changes the total by at most 10%, HF moves by at most 9%, and fastokens’ medians wobble by up to 14% in both directions, within its usual run-to-run spread. Even at the degenerate corner where the append equals the context, repair stays 2.0 $\times$ faster than retokenizing the whole text on the same engine.

The prewarmed Gigatoken column measures a different quantity than length. Its per-object cache is keyed on content, so an append costs one rescan plus the handful of pretokens it has not seen before. On this replay corpus even 100 K of new transcript is mostly repeated pretokens, so its P50 at 8 M moves by under 2% across the 100 $\times$ sweep. The price is content-dependent rather than length-dependent (novel text pays the full merge path). It comes on top of the structural properties discussed in §5.3, namely process-lifetime keyed memory and the full-context rescan that makes its floor grow with N . The sweep therefore moves the repair-versus-cache crossover rather than erasing it. At $\Delta=1$ K (the workload’s regime), repair leads from 500 K up, as in Table 2. At 5–10 K the crossover moves to about 2 M. At 50 K the prewarmed cache ties repair at 8 M (23.3 vs. 24.3 ms) and leads below it. At 100 K it leads at every shape, and fastokens too overtakes repair through the 3 M shape (10.9 ms vs. 32.2 ms at 1 M), with repair regaining the lead at 4.4 M. The mechanism is plain. Repair wins by scanning less, not by scanning faster. Once a single append is tens of times the size the workload actually produces, engines that scan faster per byte, or charge only for novel content, catch up. Within the measured append distribution, whose 99th percentile is 38 K characters, the ordering of Table 2 stands.

D Displaced Tables and the Served-Equivalent Account

Figure 16 plots the two accounts of §5.3 over the public-trace replay. One repair core climbs from 65 MB/s at 58 KB contexts to 1.35 GB/s at 1.4 MB contexts under the served account, while its scanned account holds at 1.8–3.6 MB/s.

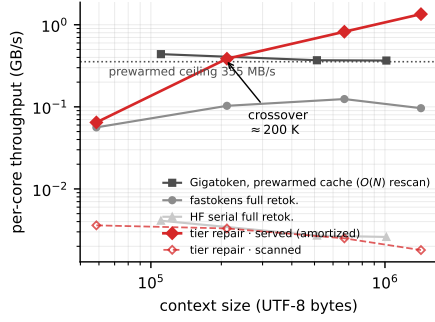


Figure 16. Per-core context throughput on the replay, two accounts kept separate (log-log). Under the served account, repair passes every $O(N)$ engine as context grows, crossing Gigatoken’s prewarmed ceiling (355 MB/s) at ~ 200 K bytes, while its scanned account stays at 1.8–3.6 MB/s.

Table 6. Guarantee ladder for lossless tokenization claims.

	Proof	Per-request check	Runtime guard
Gigatoken [27]	none	none	none
LoPT [28]	in-model [†]	length threshold	none
TokTier	in-model	splice certificate	shadow verifier

[†]our reproduction found inputs on which its published configuration escapes the theorem’s premise (§6). Its retry/fallback bounds the damage on natural text.

Table 7. Design-space position. “Exact” = token-ID-identical to the reference implementation, as established by each line of work.

	State reuse	Hardware	Exact
Fast CPU tokenizers	none	CPU	is ref. [†]
GPU tokenizers	none	GPU	no
Incremental BPE	append-only	CPU	under premise
Streaming BPE	bounded delay	CPU	merge stage
LoPT	within-request	CPU	yes
TokTier	cross-time, edits	CPU+GPU	certified

[†]or claims parity with it. §5.8 shows one widely deployed member diverging under encode history.

Table 6 places the two nearest neighbors on the guarantee ladder of §3.3, and Table 7 condenses §6 onto the two design axes the system combines.

Table 8 carries the cost model behind the rate normalization of §5.7, where, for the CPU front ends, $\mathbb{E}[t_{tok}]$ in milliseconds reads directly as CPU cores per 1,000 aggregate requests/s and the HuggingFace figure crosses 1,100 cores at $r=5$ while the tier stays in tens. Table 9 carries the five-GPU sweep behind the clock summary of §7.

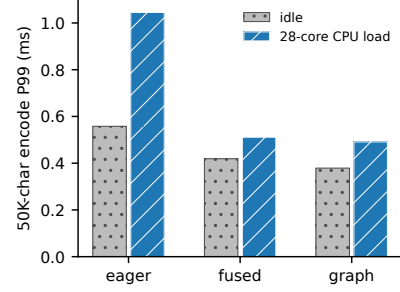


Figure 17. Host-contention companion panel to Figure 11. With 28 competing CPU processes on the host, CUDA-graph dispatch limits the P99 of a single 50 K-char full tokenization to +29% while the eager pipeline doubles (§5.4).

Table 8. Front-end cost under the measured workload mixture, the 153,529 of 153,951 collected calls with usable token accounting ($\bar{N}=117$ K tokens, $\bar{h}=0.88$). “Per 1,000 GPUs” is a rate normalization, not a cluster design, and topology, model scale, and batching enter only through r , the request completion rate per inference GPU in requests/s. The first four rows are CPU front ends, so their normalized columns are CPU cores. [†]The GPU row’s 0.15 ms is GPU service time rather than CPU time, so its normalized entries are router-GPU equivalents per 1,000 inference GPUs, not CPU cores.

Front end	$\mathbb{E}[t_{tok}]$ (ms)	cores@ $r=0.5$	@ $r=5$
HuggingFace fast	228.1	114.0	1,140
Rust (fasttokens)	13.4	6.7	67
Repair over HF engine	23.6	11.8	118
Repair over Rust engine	2.3	1.1	11
GPU full tokenization [†]	0.15	0.1	1

GPU	batch eng	e2e (GB/s)	cjk	tpl	1 M-char P50 (ms, array)
RTX 5090 (Blackwell)	4.87	4.24	4.75		0.383
RTX PRO 6000 (Blackwell)	4.22	3.58	4.09		0.381
GH200 (Hopper, Grace host)	3.91	3.11	3.79		0.464
H100 NVL (Hopper, x86 host)	3.96	3.11	3.83		3.90 [†]
A100 (Ampere)	2.45	1.69	2.56		5.64 [†]

Table 9. Cross-generation portability (Qwen3, every cell backed by a byte-identical differential check against the reference tokenizer). Throughput follows sustained clock, not SM count or bandwidth. The sweep predates the final kernel revisions behind the §5.4 headline numbers, so cross-card ordering is the comparison. [†]Python-list delivery, which includes the host-interpreter tax of §5.4.

E Serving Large Appends: Three Explored Routes

Appendix C ends with a lost regime. At $\Delta=100$ K characters the fully prewarmed cache leads repair at every context shape. Appends this large are rare, since the measured P99 is 38 K characters and 0.8% of calls reach 50 K, but a serving tier should still have an answer for them. This appendix details three routes we designed and measured to reclaim that regime. All three are exploration prototypes rather than shipped defaults, and none has passed the certification battery that gates the shipped repair path (§5.2). Each prototype does keep the exactness discipline. Every timed sample asserts its output IDs against full reference tokenization, and any divergence aborts the run. Unless stated otherwise, numbers are P50 milliseconds under the host, protocol, texts, and Qwen3 family of Appendix C ($n=24$ per cell), and window-max is the 4.4 M-character shape. Figure 18 in this appendix plots the resulting decision surface, referenced from the routing discussion of §8. This appendix carries the designs, measurements, and verdicts behind it.

E.1 Route A: parallel repair over the append (retired)

At $\Delta=100$ K the shipped repair path takes 32–40 ms (Table 10), almost entirely one serial encode of the appended text. LoPT [28] splits one request into chunks for CPU parallelism, and our splice certificate is already proved for N chunk windows (Definition A.4, Theorem A.5). Route A combines the two. It splits the append into k chunks, tokenizes them concurrently, and admits every seam with the same defensive match and certified-boundary check as deployed incremental repair. A seam that fails the check widens its overlap, and repeated failure collapses the whole append back to serial tokenization.

Two implementation facts bound the speedup. First, the reference engine holds the interpreter lock during encode. Sixteen threads tokenize eight 100 K chunks no faster than one thread (25.7 vs. 25.6 ms), so workers must be processes, and fork-preloaded workers with array transfer cost 0.3 ms of dispatch at $k=2$ and 2.0 ms at $k=16$. Second, seam matching stays serial Python at about 0.3 ms per seam. At the 1 M shape with $\Delta=100$ K, $k=8$ overlaps encode down to 7.0 ms and pays 2.1 ms of match. Moving to $k=16$ buys back 1.0 ms of encode while match doubles to 4.4 ms, and wall time regresses beyond $k=8$ at every shape. The certificate checks themselves are nearly free at 0.14–0.24 ms per repair.

The prototype works and is exact. All 1,104 timed samples match the reference IDs and byte spans. Across the full sweep, all 6,384 seams accept on the first attempt, with zero retries and zero collapses to serial. Real transcripts therefore carry certified boundaries densely enough for chunk-parallel repair. Eight cores cut $\Delta=100$ K repair to 9.8, 11.3, and 13.1 ms at 1 M, window-max, and 8 M, a 3.1–3.3 $\times$ gain over the shipped path, and simple variants (a 256-character

initial overlap, deferred text materialization) reach 11.4 ms at 8 M. Of the six cells measured here (Δ of 50 K and 100 K at the three shapes), that wins back all four window-max and 8 M cells, and both 1 M cells stay lost (9.8 vs. 2.9 ms). We retired the route anyway. Route B reaches 2.4–3.7 $\times$ lower latency on the Table 10 cells using one core instead of eight. What survives is evidence about the certificate itself. The deployed system applies one certificate across time. Route A applies many across space within a single request, at a 100% first-try acceptance rate on real text.

E.2 Route B: a cache-based tokenizer as window engine

Route A parallelizes the shipped window encoder. Route B replaces it. Gigatoken’s cache-based engine tokenizes repeated text far faster than the reference engine, but it returns token IDs only, and repair state needs byte spans (§3.4). For byte-level BPE families the byte length of every token is fixed by the vocabulary, so spans are reconstructable from IDs alone. A prefix sum over per-token byte lengths gives byte offsets, and a byte-to-character map lifts them to character spans. The reconstruction runs behind four guards. Family admission accepts byte-level BPE only. The byte-length table is derived from two independent sources and compared entry by entry. Every call checks that the reconstructed offsets exactly cover the window. For the NFC-normalized family (Qwen3), a segmented invariance guard verifies that normalization does not rewrite the window, at 0.43 ms on a 100 K window, and refuses windows it would rewrite. A refusal falls back to the reference engine. Exactness therefore comes from the tier’s own checks, not from the engine.

Reconstructed spans match the reference tokenizer’s offsets token by token on four byte-BPE families, 1.19–1.30 M tokens per family, over multilingual, code, adversarial, and real-transcript suites. We then replayed 17,066 certified splices and 1,800 adversarial edits through the full incremental repair path with the engine swapped in, in lock step with the reference engine, and observed zero divergence. The only nonzero counter is 44 guard refusals, all from one stream about Unicode transliteration whose decomposed sequences NFC rewrites, and each refusal fell back with IDs still equal. We word this as a pilot, not an admission. Admission would require the full battery of §5.2 (92,484 splices and 15,000 edits) plus engine-level integration of the fallback semantics.

The swap removes the $O(\Delta)$ bottleneck. Appended bytes tokenize at 107–128 MB/s at the 1 M and window-max shapes and 94 MB/s at 8 M, against 2.9–3.9 MB/s on the shipped path, roughly a 30–40 $\times$ higher delta term. At $\Delta=100$ K Route B takes 2.66, 3.79, and 5.34 ms at 1 M, window-max, and 8 M on one core. It matches the fully prewarmed cache at 1 M (2.86 ms) and beats it by 3.2 $\times$ at window-max and 4.4 $\times$ at 8 M, because it inherits repair’s structure and rescans nothing outside the window. Small appends do not regress, and the 1 K rows (1.5–2.2 ms) sit in the range of the shipped store’s

archived 1 K rows (1.3–6.2 ms). Two costs frame deployment. A fresh engine object takes about 180 ms to construct, outside the timing region, and a long-lived object reintroduces the content-keyed memory growth discussed in §5.3. Combining Routes A and B is mechanically sound, with every combined-run assertion passing, but pointless at these sizes. Window encodes are already sub-millisecond, so pool dispatch dominates and the $k=8$ combination stays flat at 3.1–3.2 ms (1 M) and 5.4–5.8 ms (8 M) across all append sizes.

E.3 Route C: GPU full tokenization with span export

A large append can also be served by full tokenization. The GPU path re-encodes base plus append in 1.08–1.13 ms at 1 M and 2.82–3.18 ms at window-max over both measured appends (50 K and 100 K), the cells behind the 6–28× rerouting estimate of §8, and 1.1–2.4× faster than the best CPU route in the same cell. At 8 M the picture splits by normalization. Llama 3.1 stays flat at 6.1–6.2 ms. Qwen3 requests fork on the NFC quick check (§5.4) into three service classes of about 6, 36, and 260 ms, for a pass, a CPU adjudication that confirms the text unchanged, and an actual rewrite. Sessions that hit the slow branches belong on a CPU route. The hybrid split is measured, not hypothetical.

Routing to full tokenization had a blocker. The kernels emit IDs without byte spans, so seeding session state required a reference rebuild costing 407, 2,067, and 4,112 ms at the three shapes. The rebuild row includes offset materialization and state construction, so it exceeds a bare full encode. The measured traces would even let a background rebuild hide. After a 50 K+ append, the probability that the session’s next call arrives inside the rebuild window is 0.3–4.3% across shapes, for an expected extra cost below 1.6 ms per event, at the price of one CPU core busy for 0.4–4.1 s. Span export removes the account entirely. Route B’s byte-length reconstruction runs on the device. IDs gather per-token byte lengths, a cumulative sum yields byte offsets, a second cumulative sum over non-continuation text bytes lifts them to character spans, and one packed transfer returns IDs and spans together. The shipped kernels are unchanged, the prototype reuses Route B’s guards, and exporting spans adds only 0.13–0.73 ms to the IDs-only channel up to window-max. For Qwen3 the existing NFC quick check is exactly the guard the reconstruction needs. A pass or an identity adjudication implies the spans are original-text coordinates, and a rewrite implies refusal and fallback. The measured refusal surface matches Route B’s CPU guard case for case.

Exported spans match the reference on 2,544,721 tokens across the two families, with zero ID and zero offset mismatches, and on 288 full-scale route samples of 1.06–8.1 M characters. End to end, a GPU full tokenization with span export seeds session state that then serves five real appends per session. All 345 replayed append steps and all 69 end-of-stream oracle checks equal the bootstrapped reference, and the 3 refused seedings are Qwen3 rewrite sessions at

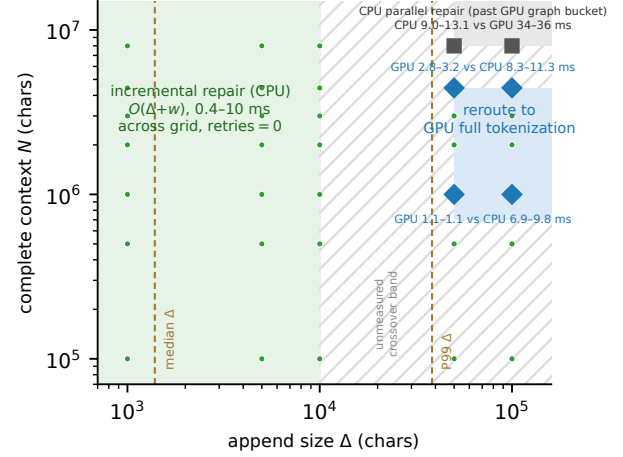


Figure 18. Measured routing phase diagram on the append-size $\times$ context plane, under exploration-prototype accounting rather than shipped defaults. Small green points are incremental repairs from the append-size sweep, and the six large markers are cells where a GPU full-tokenization rebuild and the best CPU repair option were measured head to head. Real appends concentrate far left of the GPU region, and past the GPU graph-capture bucket a parallel CPU repair prototype wins again. Hatching marks unmeasured bands.

8 M. Seeding costs 1.8–8.3 ms on the GPU plus 0.011 ms to wrap the arrays for the repair engine, about 230–500× below the reference rebuild. Building the boxed Python session state from the same arrays instead costs a further 0.2–1.0 s, a cost the Rust store form does not pay. §7 lists GPU span export as the highest-leverage missing piece of the shipped system. The prototype shows the mechanism is sound and cheap. What remains is battery-scale certification, added-token handling, and adapter-level fallback semantics. The leverage also exceeds this appendix’s tail case, because every full tokenization that seeds session state currently pays the reference rebuild.

E.4 Verdict

Table 10 settles the regime that Appendix C lost. Up to window-max the GPU route is fastest. At 8 M a CPU repair prototype wins, the parallel route at $\Delta=50$ K (9.0 ms) and the window engine at $\Delta=100$ K, with the GPU fast branch within 1.2×. Figure 18 draws the same surface with the parallel prototype as the CPU option, and this table adds Route B, which improves the 8 M cell further. Every cell the prewarmed cache won in Table 5 is reclaimed by at least one route, and the routes compose into a natural hybrid. Large appends go to the GPU, NFC slow sessions and the largest shapes go to a CPU repair route, and span export seeds session state either way.

Perspective still matters. Appends of 50 K characters and above are 0.8% of measured calls, so these routes price a tail,

Table 10. The three routes head to head at $\Delta=100$ K characters (P50 ms, Qwen3, $n=24$ per cell, same texts and protocol as Table 5, prototype accounting). “Shipped” is the deployed repair path and “cache” is Gigatoken fully prewarmed, both archived from the Appendix C sweep. Bold marks the fastest route per shape. [†]NFC quick-check fast branch. The mixed P50 over sessions that hit the slow branches is 36.3 ms (see text).

ctx	shipped	A ($k=8$)	B (1 core)	C (GPU)	cache
1 M	32.2	9.80	2.66	1.13	2.86
4.4 M	34.7	11.3	3.79	2.82	12.1
8 M	40.1	13.1	5.34	6.05 [†]	23.6

not the loop that §5 measures, and the shipped default remains repair for every session continuation until a rerouted path passes the same certification battery. What this appendix establishes is that the crossover of Appendix C is not structural. It reflects the shipped window encoder, and three independent, measured mechanisms move it.